

Lecture #3.

Pinocchio Tutorials

Kyung Hee University
Team member: Eunae Kang, Chanjin Park
Prof. Sanghyun Kim



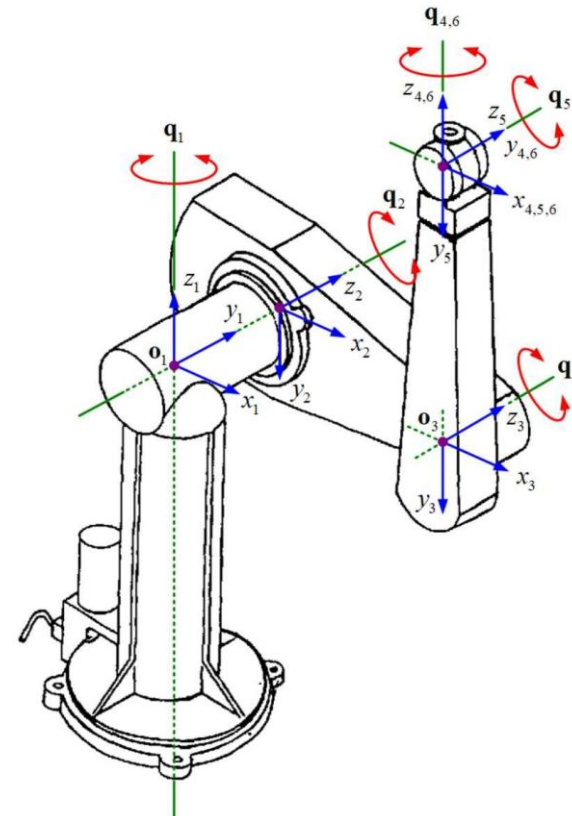
01

Geometry

Geometry

What is geometry

- Geometry refers to the actual shape and size of each robot part (link), as well as how these parts are positioned and oriented in 3D space.
- This concept is essential for kinematic and dynamic computations, collision detection, and visualization.



02

Kinematics

Rotation Matrix (SO(3))

Special Orthogonal Group

$$SO(3) = \{R \in \mathbf{R}^{3 \times 3} : RR^T = I, \det(R) = 1\}$$

Angular velocity

$$\omega = \hat{\omega}\dot{\theta}$$

Skew Symmetric Matrices

$$R = [\hat{x}|\hat{y}|\hat{z}] \rightarrow \dot{R} = [\dot{\hat{x}}|\dot{\hat{y}}|\dot{\hat{z}}] \rightarrow \dot{R} = [\omega \times \hat{x}|\omega \times \hat{y}|\omega \times \hat{z}] = \omega \times R = [\omega]R$$

$$\dot{\hat{x}} = \omega \times \hat{x}$$

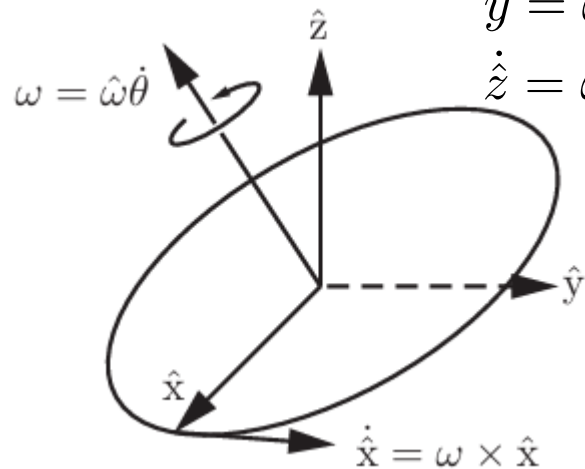
$$\dot{\hat{y}} = \omega \times \hat{y}$$

$$\dot{\hat{z}} = \omega \times \hat{z}$$

Note that $[\omega] = -[\omega]^T$

$$\omega = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix} \rightarrow [\omega] = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}$$

→ The set of all 3x3 skew-symmetric real matrices is called SO(3).



Rotation Matrix (SO(3))

Special Orthogonal Group

$$SO(3) = \{R \in \mathbf{R}^{3 \times 3} : RR^T = I, \det(R) = 1\}$$

Angular velocity

$$\omega = \hat{\omega}\dot{\theta}$$

Skew Symmetric Matrices

$$R = [\hat{x}|\hat{y}|\hat{z}] \rightarrow \dot{R} = [\dot{\hat{x}}|\dot{\hat{y}}|\dot{\hat{z}}] \rightarrow \dot{R} = [\omega \times \hat{x}|\omega \times \hat{y}|\omega \times \hat{z}] = \omega \times R = [\omega]R$$

$$\dot{\hat{x}} = \omega \times \hat{x}$$

$$\dot{\hat{y}} = \omega \times \hat{y}$$

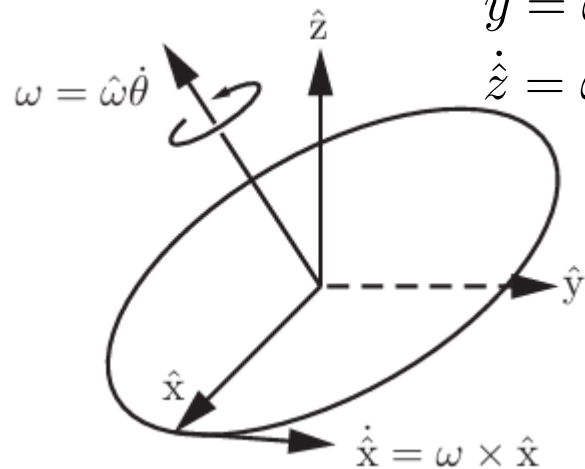
$$\dot{\hat{z}} = \omega \times \hat{z}$$

$$\Rightarrow \dot{R}R^{-1} = [\omega_s]$$

Let ω_b be expressed ω in body-frame coordinates.

$$\omega_b = R_{sb}^{-1}\omega_s = R^{-1}\omega_s = R^T\omega_s$$

$$\Rightarrow R^{-1}\dot{R} = [\omega_b]$$



Rotation Matrix (SO(3))

Linear Differential Equations

$$x \in \mathbf{R} : \dot{x}(t) = ax(t) \rightarrow x(t) = e^{at}x(0) \Rightarrow x \in \mathbf{R}^n : \dot{x}(t) = Ax(t) \rightarrow x(t) = e^{At}x(0)$$

Taylor series

$$e^{at} = 1 + at + \frac{(at)^2}{2!} + \frac{(at)^3}{3!} + \dots \Rightarrow e^{At} = I + At + \frac{(At)^2}{2!} + \frac{(At)^3}{3!} + \dots$$

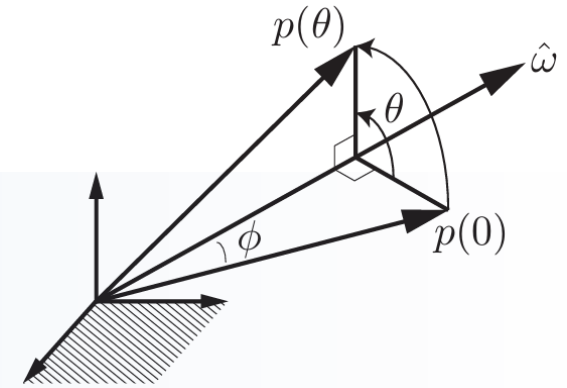
$$\begin{aligned} \bullet \quad \dot{x}(t) &= \left(\frac{d}{dt} e^{At} \right) x(0) \\ &= \frac{d}{dt} \left(I + At + \frac{(At)^2}{2!} + \frac{(At)^3}{3!} + \dots \right) x(0) \\ &= \left(A + A^2t + \frac{A^3t^2}{2!} + \dots \right) x(0) \\ &= Ae^{At}x(0) = Ax(t) \end{aligned}$$

Rotation Matrix (SO(3))

Exponential Coordinates of Rotation

$$\dot{p}(t) = \hat{\omega} \times p(t) = [\hat{\omega}]p(t)$$

$$p(t) = e^{[\hat{\omega}]t} p(0) \rightarrow p(\theta) = e^{[\hat{\omega}]\theta} p(0)$$



Rodrigues' Formula

$$\text{Rot}(\hat{\omega}, \theta) = e^{[\hat{\omega}]\theta} = I + \sin\theta[\hat{\omega}] + (1 - \cos\theta)[\hat{\omega}]^2 \in SO(3)$$

Homogeneous Transformation Matrix (SE(3))

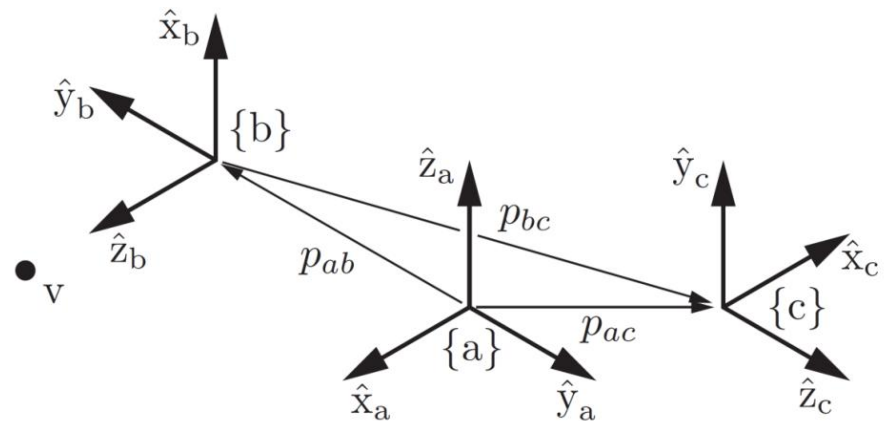
Special Euclidean Group

$$SE(3) = \{(R, p) : R \in SO(3), p \in \mathbf{R}^3\} = SO(3) \times \mathbf{R}^3$$

The SE(3) is the set of all 4x4 real matrices T of the form

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}, \text{ where } R \in SO(3) \text{ and } p \in \mathbf{R}^3 \text{ is a column vector.}$$

$$T^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix} \in SE(3)$$



Twists

Remind, $\dot{R}R^{-1} = [\omega_s]$

$$R^{-1}\dot{R} = [\omega_b]$$

Let us see what happens when we pre-multiply $\dot{T}$ by T .

$$\begin{aligned} T^{-1}\dot{T} &= \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \dot{R} & \dot{p} \\ 0 & 0 \end{bmatrix} \\ &= \begin{bmatrix} R^T \dot{R} & R^T \dot{p} \\ 0 & 0 \end{bmatrix} \\ &= \begin{bmatrix} [\omega_b] & v_b \\ 0 & 0 \end{bmatrix} \end{aligned}$$

We assemble ω_b and v_b into a six-dimensional twist,

$$\nu_b = \begin{bmatrix} \omega_b \\ v_b \end{bmatrix} \in \mathbf{R}^6, [\nu_b] = \begin{bmatrix} [\omega_b] & v_b \\ 0 & 0 \end{bmatrix} = T^{-1}\dot{T} \in se(3)$$

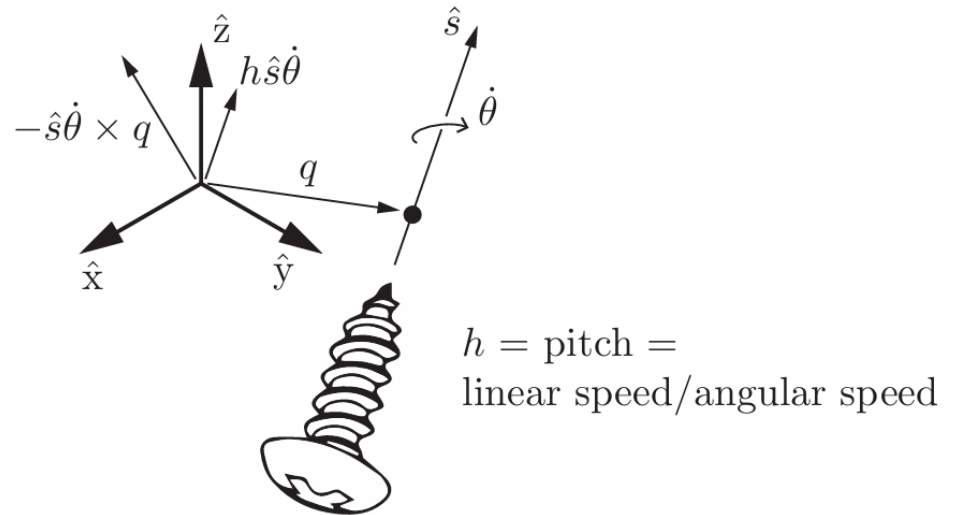
$$\nu_s = \begin{bmatrix} \omega_s \\ v_s \end{bmatrix} \in \mathbf{R}^6, [\nu_s] = \begin{bmatrix} [\omega_s] & v_s \\ 0 & 0 \end{bmatrix} = \dot{T}T^{-1} \in se(3)$$

Screw axis

$$\nu = \begin{bmatrix} \omega \\ v \end{bmatrix} = S\dot{\theta}$$

$$S = \begin{bmatrix} \omega \\ v \end{bmatrix} \in \mathbf{R}^6$$

$$e^{[S]\theta} = \begin{bmatrix} e^{[\omega]\theta} & (I\theta + (1 - \cos\theta)[\omega] + (\theta - \sin\theta)[\omega]^2)v \\ 0 & 1 \end{bmatrix} \in SE(3)$$



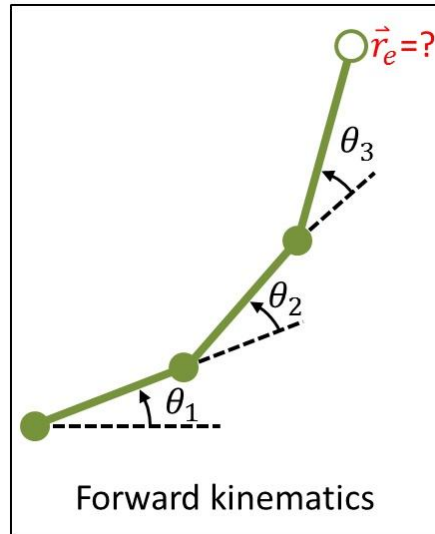
Kinematics

What is kinematics

- Kinematics refers to the study of motion of objects (especially robots) without considering the forces that cause the motion.

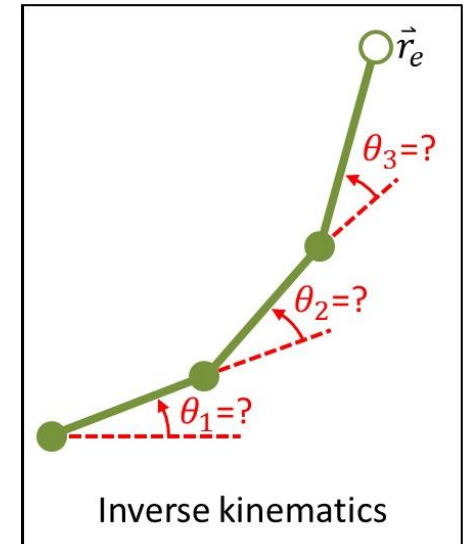
Forward Kinematics

- Given the joint angles or positions, calculate the position and orientation of the robot's end-effector or any other links.
- Given θ , find $T(\theta) \in SE(3)$.



Inverse Kinematics

- Given the desired position and orientation of the end-effector, determine the necessary joint angles or positions to achieve it.
- Given $X \in SE(3)$, find θ such that $T(\theta) = X$.



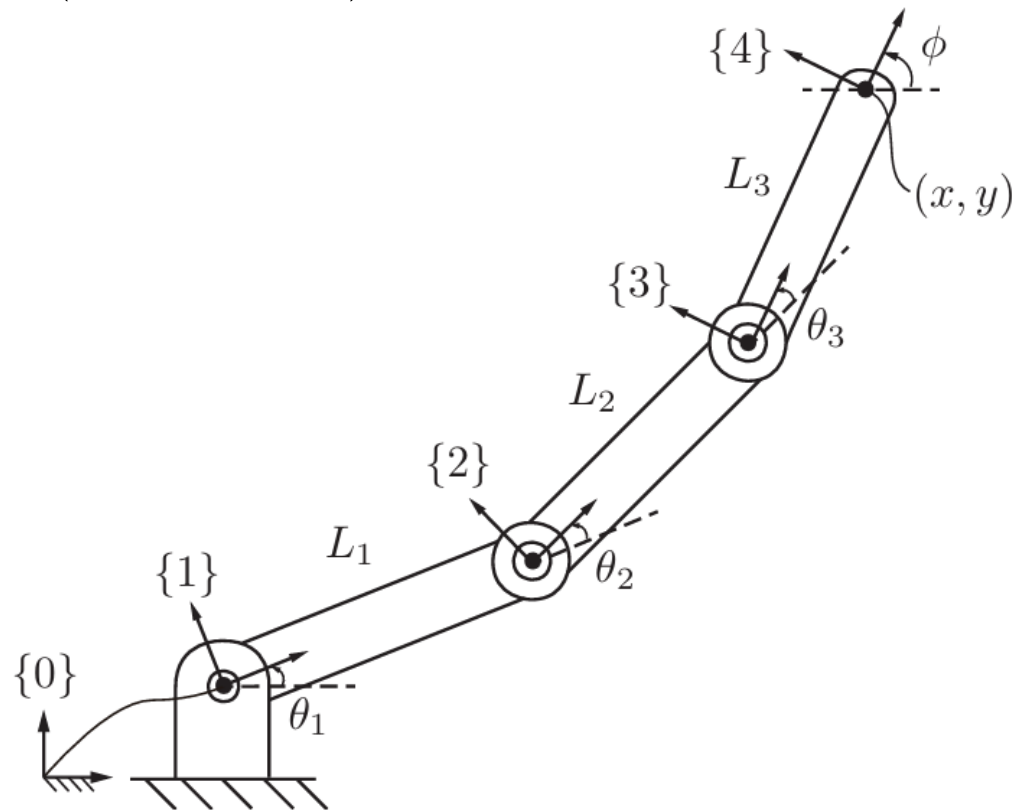
Kinematics | Forward Kinematics

- The forward kinematics of a robot refers to the calculation of the position and orientation of its end-effector frame from its joint coordinates.

$$x = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) + L_3 \cos(\theta_1 + \theta_2 + \theta_3)$$

$$y = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2) + L_3 \sin(\theta_1 + \theta_2 + \theta_3)$$

$$\phi = \theta_1 + \theta_2 + \theta_3$$



Kinematics | Forward Kinematics

- The forward kinematics of a robot refers to the calculation of the position and orientation of its end-effector frame from its joint coordinates.

$$T_{04} = T_{01}T_{12}T_{23}T_{34}$$

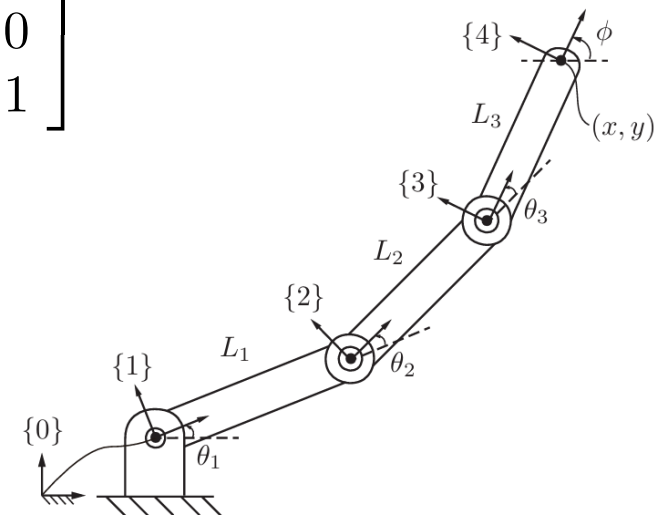
$$T_{01} = \begin{bmatrix} \cos\theta_1 & -\sin\theta_1 & 0 & 0 \\ \sin\theta_1 & \cos\theta_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{12} = \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 & 0 & L_1 \\ \sin\theta_2 & \cos\theta_2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{23} = \begin{bmatrix} \cos\theta_3 & -\sin\theta_3 & 0 & L_2 \\ \sin\theta_3 & \cos\theta_3 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{34} = \begin{bmatrix} 1 & 0 & 0 & L_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M = \begin{bmatrix} 1 & 0 & 0 & L_1 + L_2 + L_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Kinematics | Forward Kinematics

- The forward kinematics of a robot refers to the calculation of the position and orientation of its end-effector frame from its joint coordinates.

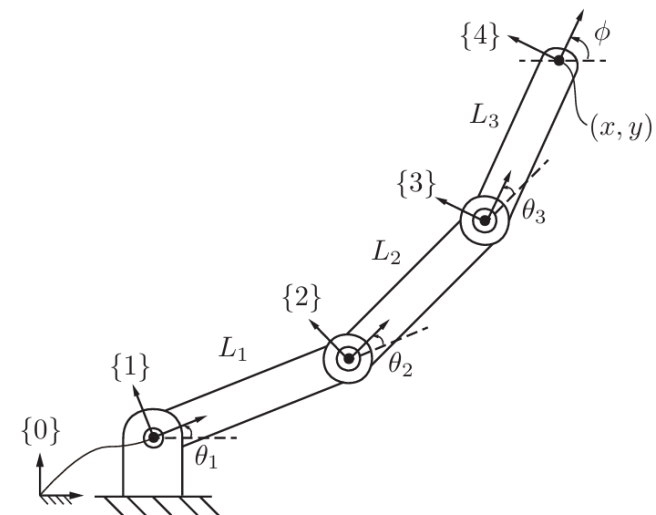
$$\omega_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad q_3 = \begin{bmatrix} L_1 + L_2 \\ 0 \\ 0 \end{bmatrix} \rightarrow v_3 = -\omega_3 \times q_3$$

$$S_3 = \begin{bmatrix} \omega_3 \\ v_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ -(L_1 + L_2) \\ 0 \end{bmatrix}, \quad [S_3] = \begin{bmatrix} [\omega] & v \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & -(L_1 + L_2) \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\rightarrow T_{04} = T_{01}T_{12}T_{23}T_{34} = e^{[S_1]\theta_1}e^{[S_2]\theta_2}e^{[S_3]\theta_3}M$$

Given θ , evaluate the product of exponentials formula in the space frame.

$$T_\theta = e^{[S_1]\theta_1}e^{[S_2]\theta_2} \dots e^{[S_n]\theta_n}M$$



Numerical Inverse Kinematics

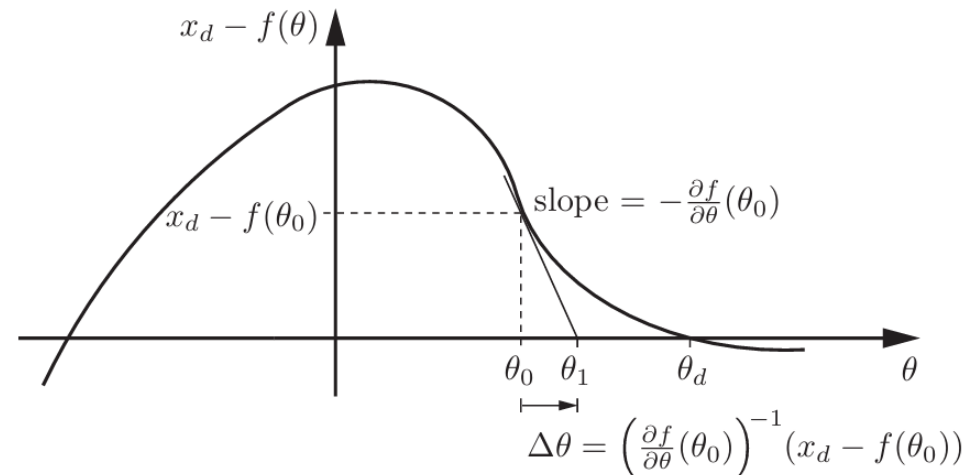
Kinematics Newton Raphson Method

Given θ^0 which is “close to” a solution θ_d , the kinematics can be expressed as the Taylor expansion.

$$x_d = f(\theta_d) = f(\theta^i) + \left. \frac{\partial f}{\partial \theta} \right|_{\theta^0} (\theta_d - \theta^0) + \text{h.o.t}$$

$$\rightarrow x_d - f(\theta^i) = J(\theta^i) \Delta \theta$$

$$\rightarrow J^{-1}(\theta^i)(x_d - f(\theta^i)) = \Delta \theta$$



Numerical Inverse Kinematics

Kinematics Newton Raphson Method

Given θ^0 which is “close to” a solution θ_d , the kinematics can be expressed as the Taylor expansion.

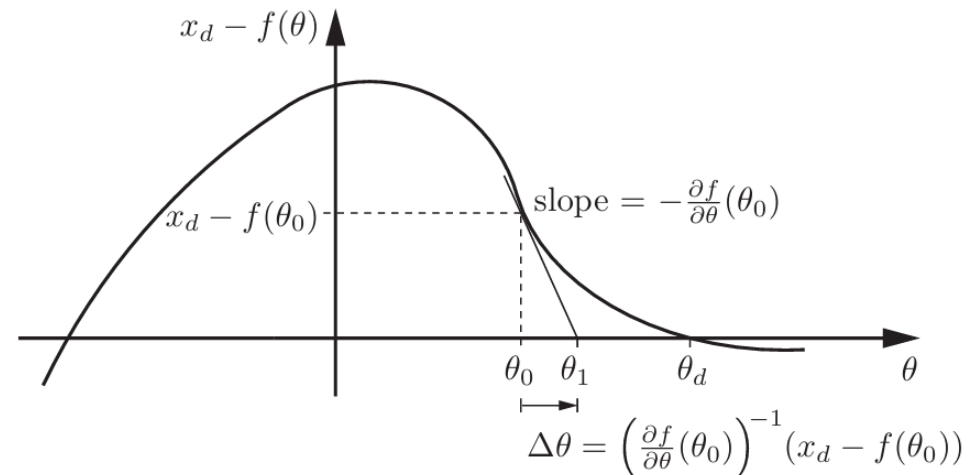
$$x_d = f(\theta_d) = f(\theta^i) + \left. \frac{\partial f}{\partial \theta} \right|_{\theta^0} (\theta_d - \theta^0) + \text{h.o.t}$$

$$\rightarrow x_d - f(\theta^i) = J(\theta^i) \Delta \theta$$

$$\rightarrow J^{-1}(\theta^i)(x_d - f(\theta^i)) = \Delta \theta$$

What if J is not invertible? By replacing J^{-1} with the pseudoinverse of J .

$$J^\dagger(\theta^i)(x_d - f(\theta^i)) = \Delta \theta^*$$



03

Dynamics (1)

Learning Objectives

- Understand the equation of motion for robotic systems
- Explore torques applied by actuators and external forces on manipulators
- Inverse Dynamics: Finding joint torque vector τ knowing trajectory points $q, \dot{q}, \ddot{q}$; effectively used to manipulator control.
- Forward Dynamics: Finding joint acceleration $\ddot{q}$ knowing $q, \dot{q}, \tau$; used in simulator
- Assertion through Pinocchio library

Inverse Dynamics | Recursive Newton–Euler Algorithm

■ The inverse dynamics can be calculated in three steps as follows:

- Calculate the velocity and acceleration of each link.
- Calculate the forces required to produce these accelerations.
- Calculate the forces transmitted across the joints from the forces acting on the links.

Inverse Dynamics | Recursive Newton–Euler Algorithm

- ▶ Calculate the velocity and acceleration of each link.

$$\omega_i = {}^i E_{i-1} \omega_{i-1} + z \dot{q}_i$$

$$v_i = {}^i E_{i-1} (v_{i-1} + \omega_{i-1} \times {}^{i-1} r_i)$$

$$\dot{\omega}_i = {}^i E_{i-1} \dot{\omega}_{i-1} + z \ddot{q}_i + ({}^i E_{i-1} \omega_{i-1}) \times z \dot{q}_i$$

$$\dot{v}_i = {}^i E_{i-1} (\dot{v}_{i-1} + \dot{\omega}_{i-1} \times {}^{i-1} r_i + \omega_{i-1} \times \omega_{i-1} \times {}^{i-1} r_i)$$

- ▶ Symbols:

$\omega_i, \dot{\omega}_i$ angular velocity and acceleration of body i

$v_i, \dot{v}_i$ linear velocity and acceleration of the origin of F_i

${}^i E_{i-1}$ 3×3 orthogonal rotation matrix transforming from F_{i-1} to F_i coordinates

${}^{i-1} r_i$ position of origin of F_i relative to F_{i-1} , expressed in F_{i-1} coordinates

$\dot{q}_i, \ddot{q}_i$ joint i velocity and acceleration

z direction of joint i axis in F_i coordinates

Inverse Dynamics | Recursive Newton–Euler Algorithm

- ▶ Calculate the forces required to produce these accelerations.

$$F_i = m_i(\dot{v}_i + \dot{\omega}_i \times c_i + \omega_i \times \omega_i \times c_i)$$

$$N_i = I_i^{cm} \dot{\omega}_i + \omega_i \times I_i^{cm} \omega_i$$

- ▶ Symbols:

F_i, N_i net force and moment on body i , expressed at its center of mass

m_i, c_i, I_i^{cm} mass, center of mass, and rotational inertia about center of mass for body i in F_i coordinates

Inverse Dynamics | Recursive Newton–Euler Algorithm

- ▶ Calculate the forces transmitted across the joints from the forces acting on the links.

$$f_i = F_i + {}^i E_{i+1} f_{i+1}$$

$$n_i = N_i + c_i \times F_i + {}^i E_{i+1} n_{i+1} + {}^i r_{i+1} \times {}^i E_{i+1} f_{i+1}$$

$$\tau_i = z^T n_i$$

- ▶ Symbols:

f_i, n_i force and moment exerted on body i through joint i

τ_i joint i force variables

z direction of joint i axis in F_i coordinates

Inverse Dynamics | Recursive Newton–Euler Algorithm

- Derive the dynamic equations for a planar 2R open chain moving in the presence of gravity. The chain moves in the $\hat{x} - \hat{y}$ plane, with gravity g acting in the $-\hat{y}$ -direction. The two links are modeled as point masses m_1 and m_2 concentrated at the ends of each link.

$$c_1 = L_1 \hat{X}_1$$

$$c_2 = L_2 \hat{X}_2$$

- Due to the assumption of point mass,

$$I_1^{cm} = 0$$

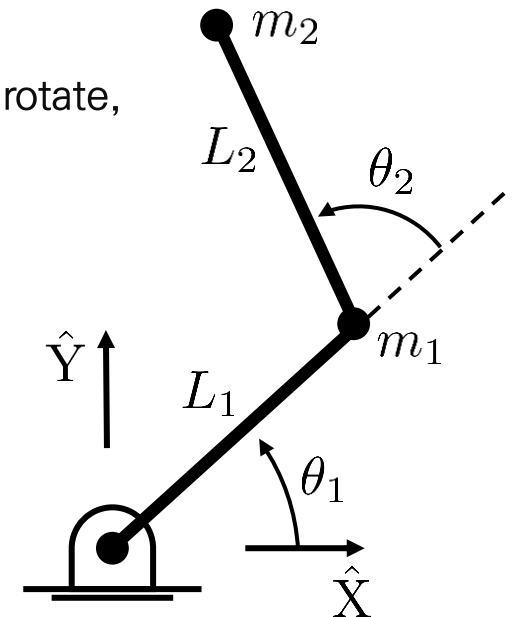
$$I_2^{cm} = 0$$

- If there is no force acting on the end-effector and the base does not rotate,

$$f_3 = 0 \quad \omega_0 = 0 \quad \dot{v}_0 = g \hat{Y}_0$$

$$n_3 = 0 \quad \dot{\omega}_0 = 0$$

The robot's base accelerating upward with g creates a **virtual gravity effect**, making the robot experience forces identical to those caused by gravity acting downward.



Inverse Dynamics | Recursive Newton–Euler Algorithm

- The rotation matrices transforming between F_{i-1} and F_i coordinate is,

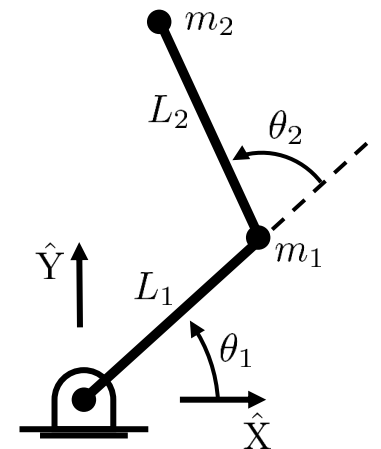
$${}^i E_{i-1} = \begin{bmatrix} c_i & s_i & 0.0 \\ -s_i & c_i & 0.0 \\ 0.0 & 0.0 & 1.0 \end{bmatrix} \quad {}^{i-1} E_i = \begin{bmatrix} c_i & -s_i & 0.0 \\ s_i & c_i & 0.0 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}$$

- Forward iteration for link 1 is,

$$\omega_1 = \dot{\theta}_1 \hat{Z}_1 = \begin{bmatrix} 0 \\ 0 \\ \dot{\theta}_1 \end{bmatrix} \quad F_1 = m_1 \left(\begin{bmatrix} 0 \\ L_1 \ddot{\theta}_1 \\ 0 \end{bmatrix} + \begin{bmatrix} -L_1 \dot{\theta}_1^2 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} gs_1 \\ gc_1 \\ 0 \end{bmatrix} \right)$$

$$\dot{\omega}_1 = \ddot{\theta}_1 \hat{Z}_1 = \begin{bmatrix} 0 \\ 0 \\ \ddot{\theta}_1 \end{bmatrix} = \begin{bmatrix} -m_1 L_1 \dot{\theta}_1^2 + m_1 gs_1 \\ m_1 L_1 \ddot{\theta}_1 + m_1 gc_1 \\ 0 \end{bmatrix}$$

$$\dot{v}_1 = \begin{bmatrix} c_1 & s_1 & 0 \\ -s_1 & c_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ g \\ 0 \end{bmatrix} = \begin{bmatrix} gs_1 \\ gc_1 \\ 0 \end{bmatrix} \quad N_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$



Inverse Dynamics | Recursive Newton-Euler Algorithm

Forward iteration for link 2 is,

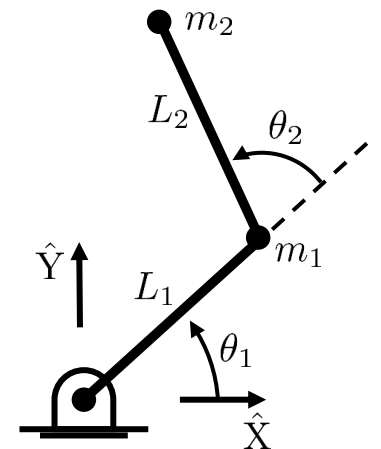
$$\omega_2 = \begin{bmatrix} 0 \\ 0 \\ \dot{\theta}_1 + \dot{\theta}_2 \end{bmatrix} \quad \dot{\omega}_1 = \begin{bmatrix} 0 \\ 0 \\ \ddot{\theta}_1 + \ddot{\theta}_2 \end{bmatrix}$$

$$\dot{v}_1 = \begin{bmatrix} c_2 & s_2 & 0 \\ -s_2 & c_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -L_1\dot{\theta}_1^2 + gs_1 \\ L_1\ddot{\theta}_1 + gc_1 \\ 0 \end{bmatrix} = \begin{bmatrix} L_1\ddot{\theta}_1 s_2 - L_1\dot{\theta}_1^2 c_2 + gs_{12} \\ L_1\ddot{\theta}_1 c_2 + L_1\dot{\theta}_1^2 s_2 + gc_{12} \\ 0 \end{bmatrix}$$

$$F_2 = m_2 \left(\begin{bmatrix} 0 \\ L_2(\ddot{\theta}_1 + \ddot{\theta}_2) \\ 0 \end{bmatrix} + \begin{bmatrix} -L_2(\dot{\theta}_1 + \dot{\theta}_2)^2 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} L_1\ddot{\theta}_1 s_2 - L_1\dot{\theta}_1^2 c_2 + gs_{12} \\ L_1\ddot{\theta}_1 c_2 + L_1\dot{\theta}_1^2 s_2 + gc_{12} \\ 0 \end{bmatrix} \right)$$

$$= \begin{bmatrix} m_2 L_1 \ddot{\theta}_1 s_2 - m_2 L_1 \dot{\theta}_1^2 c_2 + m_2 g s_{12} - m_2 L_2 (\dot{\theta}_1 + \dot{\theta}_2)^2 \\ m_2 L_1 \ddot{\theta}_1 c_2 + m_2 L_1 \dot{\theta}_1^2 s_2 + m_2 g c_{12} + m_2 L_2 (\ddot{\theta}_1 + \ddot{\theta}_2) \\ 0 \end{bmatrix}$$

$$N_2 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$



Inverse Dynamics | Recursive Newton–Euler Algorithm

➤ Backward iteration for link 2 is,

$$f_2 = F_2$$
$$n_2 = \begin{bmatrix} 0 \\ 0 \\ m_2 L_1 L_2 c_2 \ddot{\theta}_1 + m_2 L_1 L_2 s_2 \dot{\theta}_1^2 + m_2 L_2 g c_{12} + m_2 L_2^2 (\ddot{\theta}_1 + \ddot{\theta}_2) \end{bmatrix}$$

➤ Backward iteration for link 1 is,

$$f_1 = \begin{bmatrix} c_2 & -s_2 & 0 \\ s_2 & c_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} m_2 L_1 s_2 \ddot{\theta}_1 - m_2 L_1 c_2 \dot{\theta}_1^2 + m_2 g s_{12} - m_2 L_2 (\dot{\theta}_1 + \dot{\theta}_2)^2 \\ m_2 L_1 c_2 \ddot{\theta}_1 + m_2 L_1 s_2 \dot{\theta}_1^2 + m_2 g c_{12} + m_2 L_2 (\ddot{\theta}_1 + \ddot{\theta}_2) \\ 0 \end{bmatrix}$$
$$+ \begin{bmatrix} -m_1 L_1 \dot{\theta}_1^2 + m_1 g s_1 \\ m_1 L_1 \ddot{\theta}_1 + m_1 g c_1 \\ 0 \end{bmatrix}$$
$$n_1 = \begin{bmatrix} 0 \\ 0 \\ m_2 L_1 L_2 c_2 \ddot{\theta}_1 + m_2 L_1 L_2 s_2 \dot{\theta}_1^2 + m_2 L_2 g c_{12} + m_2 L_2^2 (\ddot{\theta}_1 + \ddot{\theta}_2) \end{bmatrix}$$
$$+ \begin{bmatrix} 0 \\ 0 \\ m_1 L_1^2 \ddot{\theta}_1 + m_1 L_1 g c_1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ m_2 L_1^2 \ddot{\theta}_1 - m_2 L_1 L_2 s_2 (\dot{\theta}_1 + \dot{\theta}_2)^2 + m_2 L_1 g s_2 s_{12} \\ + m_2 L_1 L_2 c_2 (\ddot{\theta}_1 + \ddot{\theta}_2) + m_2 L_1 g c_2 c_{12} \end{bmatrix}$$

Inverse Dynamics | Recursive Newton–Euler Algorithm

• The joint torque is the Z component of n_i

$$\tau_1 = m_2 L_2^2 (\ddot{\theta}_1 + \ddot{\theta}_2) + m_2 L_1 L_2 c_2 (2\ddot{\theta}_1 + \ddot{\theta}_2) + (m_1 + m_2) L_1^2 \ddot{\theta}_1 - m_2 L_1 L_2 s_2 \dot{\theta}_2^2 \\ - 2m_2 L_1 L_2 s_2 \dot{\theta}_1 \dot{\theta}_2 + m_2 L_2 g c_{12} + (m_1 + m_2) L_1 g c_1$$

$$\tau_2 = m_2 L_1 L_2 c_2 \ddot{\theta}_1 + m_2 L_1 L_2 s_2 \dot{\theta}_1^2 + m_2 L_2 g c_{12} + m_2 L_2^2 (\ddot{\theta}_1 + \ddot{\theta}_2)$$

$$\tau = M(\theta)\ddot{\theta} + C(\theta, \dot{\theta}) + G(\theta)$$

Mass matrix represents the system's inertia, relating joint accelerations to applied torques

Centripetal terms terms containing $\dot{\theta}_i^2$ account for forces due to the rotation of the links

Coriolis terms terms containing $\dot{\theta}_i \dot{\theta}_j, i \neq j$ represent forces caused by the rotation of the system

Gravity terms describe the forces acting on the system due to gravity, depending on joint positions

Inverse Dynamics | Recursive Newton–Euler Algorithm

Algorithm:

$$\mathbf{v}_0 = \mathbf{0}$$

$$\mathbf{a}_0 = -\mathbf{a}_g$$

for $i = 1$ to N_B do

$$[\mathbf{X}_J, \mathbf{S}_i, \mathbf{v}_J, \mathbf{c}_J] = \text{jcalc}(\text{jtype}(i), \mathbf{q}_i, \dot{\mathbf{q}}_i)$$

$${}^i\mathbf{X}_{\lambda(i)} = \mathbf{X}_J \mathbf{X}_T(i)$$

if $\lambda(i) \neq 0$ then

$${}^i\mathbf{X}_0 = {}^i\mathbf{X}_{\lambda(i)} {}^{\lambda(i)}\mathbf{X}_0$$

end

$$\mathbf{v}_i = {}^i\mathbf{X}_{\lambda(i)} \mathbf{v}_{\lambda(i)} + \mathbf{v}_J$$

$$\mathbf{a}_i = {}^i\mathbf{X}_{\lambda(i)} \mathbf{a}_{\lambda(i)} + \mathbf{S}_i \ddot{\mathbf{q}}_i + \mathbf{c}_J + \mathbf{v}_i \times \mathbf{v}_J$$

$$\mathbf{f}_i = \mathbf{I}_i \mathbf{a}_i + \mathbf{v}_i \times^* \mathbf{I}_i \mathbf{v}_i - {}^i\mathbf{X}_0^* \mathbf{f}_i^x$$

end

for $i = N_B$ to 1 do

$$\boldsymbol{\tau}_i = \mathbf{S}_i^T \mathbf{f}_i$$

if $\lambda(i) \neq 0$ then

$$\mathbf{f}_{\lambda(i)} = \mathbf{f}_{\lambda(i)} + {}^{\lambda(i)}\mathbf{X}_i^* \mathbf{f}_i$$

end

end

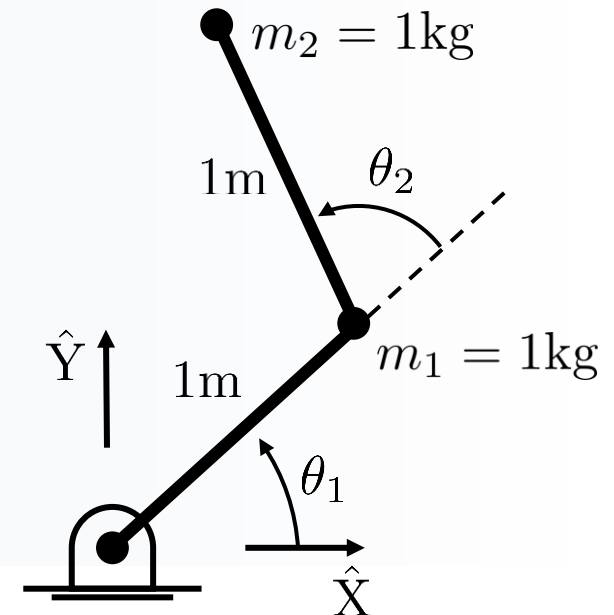
pin, rnea(model, data, q, v, a)

- Computes inverse dynamics torques using the Recursive Newton–Euler Algorithm. Used to find the required joint torques to track a desired motion.

Inverse Dynamics | Assertion

Defining Point Mass in URDF

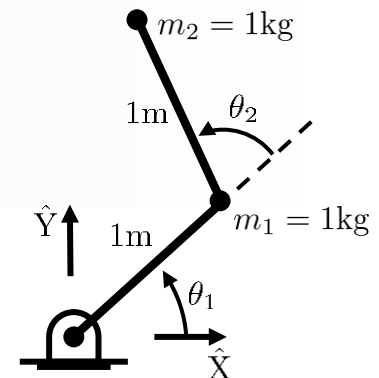
```
<?xml version="1.0"?>
<robot name="double_pendulum_point-mass">
  <link name="base_link">
  </link>
  <link name="link1">
    <inertial>
      <!-- link1's COM at (l1, 0, 0) -->
      <origin xyz="1.0 0 0" rpy="0 0 0"/>
      <mass value="1.0"/>
      <inertia ixx="0" ixy="0" ixz="0"
                iyy="0" iyz="0" izz="0"/>
    </inertial>
  </link>
  <link name="link2">
    <inertial>
      <!-- link2's COM at (l2, 0, 0) -->
      <origin xyz="1.0 0 0" rpy="0 0 0"/>
      <mass value="1.0"/>
      <inertia ixx="0" ixy="0" ixz="0"
                iyy="0" iyz="0" izz="0"/>
    </inertial>
  </link>
  ...
</robot>
```



Inverse Dynamics | Assertion

Defining Point Mass in URDF

```
...  
<joint name="joint1" type="revolute">  
  <parent link="base_link"/>  
  <child link="link1"/>  
  <origin xyz="0 0 0" rpy="0 0 0"/>  
  <axis xyz="0 1 0"/>  
  <limit lower="-3.14159" upper="3.14159" effort="100.0" velocity="100.0"/>  
</joint>  
  
<joint name="joint2" type="revolute">  
  <parent link="link1"/>  
  <child link="link2"/>  
  <origin xyz="1.0 0 0" rpy="0 0 0"/>  
  <axis xyz="0 1 0"/>  
  <limit lower="-3.14159" upper="3.14159" effort="100.0" velocity="100.0"/>  
</joint>  
  
</robot>
```



Inverse Dynamics | Assertion

inverse-dynamics-pm.py

```
data = model.createData()

print("nq =", model.nq, "nv =", model.nv)

q = np.array([0.5, -0.3])
v = np.array([0.1, 0.2])
a = np.array([0.0, 1.0])

tau = pin.rnea(model, data, q, v, a)
```

```
(pin) home@DESKTOP-T4D06E6:~/pinocchio/examples$ python inverse-
dynamics-pm.py
nq = 2 nv = 2
Joint torques (tau): [-24.85364489 -8.61740833]
```

Inverse Dynamics | Assertion

Initial conditions

```
q = np.array([0.5, -0.3])  
v = np.array([0.1, 0.2])  
a = np.array([0.0, 1.0])
```

pin, rnea(model, data, q, v, a)

Joint torques (tau): [-24.85364489 -8.61740833]

Assertion

$c_1 = 0.8776$ $s_1 = 0.4794$ $c_2 = 0.9553$ $s_2 = -0.2955$
 $c_{12} = 0.98$

Joint torques (tau): [-24.853372 -8.616755]

Forward Dynamics | Articulated-Body Algorithm

■ Purpose of the algorithm

- Computes joint accelerations $\ddot{q}$ from joint torques τ , joint positions q , and velocities $\dot{q}$
- Provides an $O(n)$ forward dynamics solution without explicitly forming or inverting the mass matrix $M(q)$

■ Key Idea

- Propagates dynamics through the kinematic tree using recursive passes.
- Avoids computing $M(q)$ or $M^{-1}(q)$; instead computes accelerations directly

Forward Dynamics | Articulated-Body Algorithm

Articulated body A_i

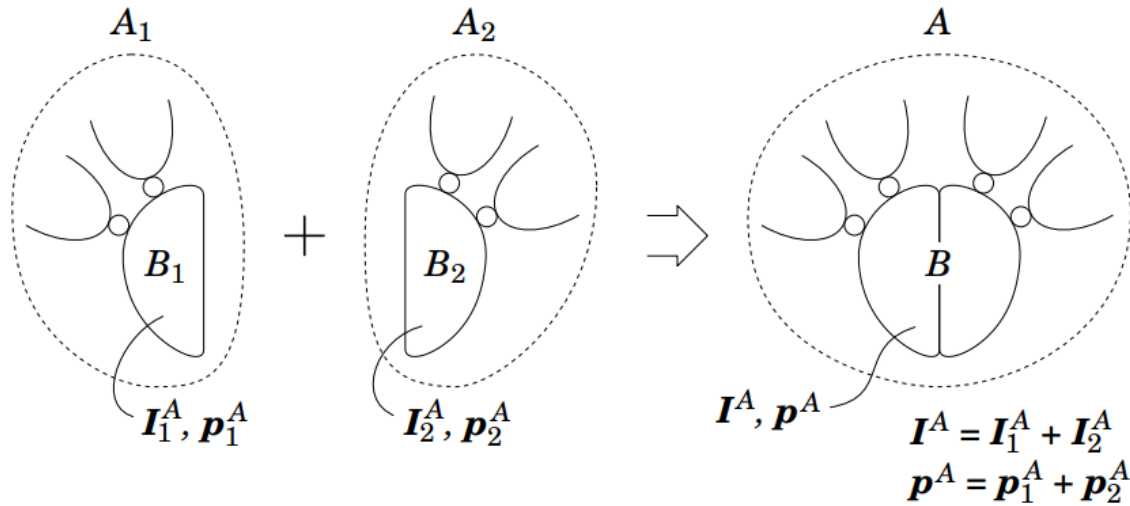


Figure 7.2: Addition of articulated-body inertias and bias forces

- For a tree-structured robot, the subtree rooted at link i is called the articulated body A_i
- the articulated-body inertia I_i^A , representing the effective inertia of body i plus all descendants.
- the articulated-body bias force p_i^A , combining centrifugal, Coriolis, gravity, and external forces from the entire subtree.

Forward Dynamics | Articulated-Body Algorithm

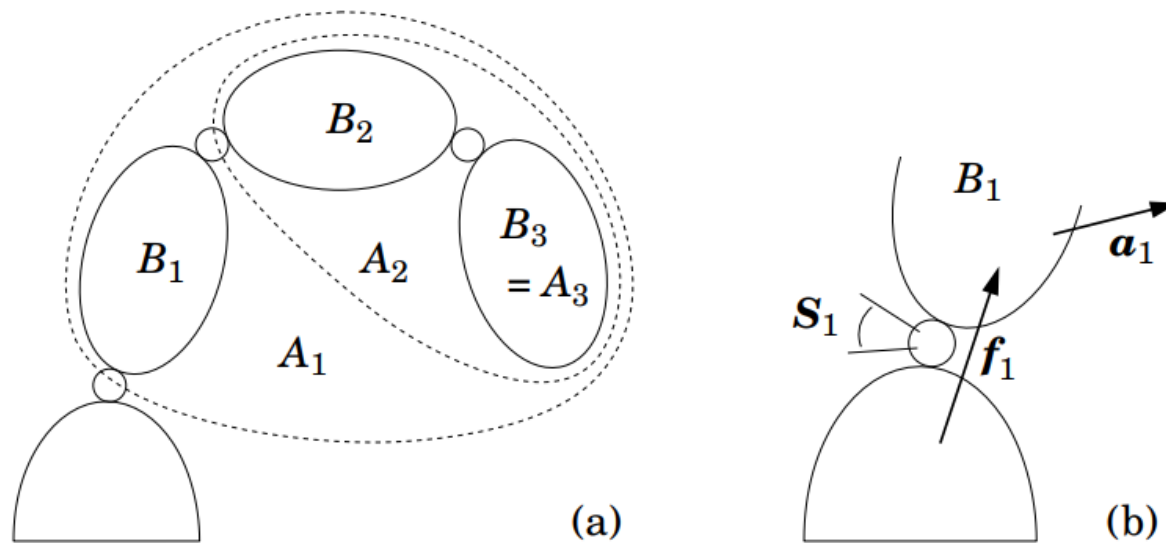


Figure 7.5: Aspects of the articulated-body algorithm: (a) defining the articulated bodies, and (b) solving for the acceleration of joint 1

- Joint i is not included in A_i ; only bodies are.
- Body i (denoted B_i) is the handle of the articulated body A_i

Forward Dynamics | Articulated-Body Algorithm

Equation of motion for the articulated body A_i

- ▶ The force transmitted through joint i :

$$f_i = I_i^A a_i + p_i^A$$

(articulated-body inertia $\times$ acceleration + articulated-body bias force)

Joint constraint on acceleration

- ▶ The acceleration of body B_i must satisfy

$$a_i = a_{\lambda(i)} + c_i + S_i \ddot{q}_i$$

where

- ▶ $a_{\lambda(i)}$ acceleration of parent body
- ▶ c_i motion-dependent bias term
- ▶ S_i joint motion subspace

Forward Dynamics | Articulated-Body Algorithm

Joint constraint on transmitted force

- ▶ The torque at the joint is the projection of transmitted force:

$$S_i^T f_i = \tau_i$$

Solving for Joint Acceleration $\ddot{q}_i$

- ▶ Substitute a_i into the force equation:

$$f_i = I_i^A(a_{\lambda(i)} + c_i + S_i \ddot{q}_i) + p_i^A$$

- ▶ Apply joint constraint $S_i^T f_i = \tau_i$:

$$S_i^T [I_i^A(a_{\lambda(i)} + c_i + S_i \ddot{q}_i) + p_i^A] = \tau_i$$

- ▶ Rearranged to solve for $\ddot{q}_i$:

$$\ddot{q}_i = (S_i^T I_i^A S_i)^{-1} [\tau_i - S_i^T I_i^A (a_{\lambda(i)} + c_i) - S_i^T p_i^A]$$

Forward Dynamics | Articulated-Body Algorithm

■ The forward dynamics can be calculated in three steps as follows:

- Calculate the velocity-product accelerations, and the rigid-body bias forces.
- Calculate the articulated-body inertias and bias forces.
- Calculate the accelerations.

Forward Dynamics | Articulated-Body Algorithm

- ▶ Calculate the velocity-product accelerations, and the rigid-body bias forces.

$$v_{Ji} = S_i \dot{q}_i$$

$$c_{Ji} = \dot{S}_i \dot{q}_i$$

$$v_i = v_{\lambda(i)} + v_{Ji} \quad (v_0 = 0)$$

$$c_i = c_{Ji} + v_i \times v_{Ji}$$

$$p_i = v_i \times^* I_i v_i - f_i^x$$

- ▶ Symbols:

S_i Joint motion subspace matrix

v_i, c_i The body velocities and the velocity-product accelerations

p_i The rigid-body bias forces, for use in later passes

Forward Dynamics | Articulated-Body Algorithm

- ▶ Calculate the articulated-body inertias and bias forces.

$$I_i^A = I_i + \sum_{j \in \mu(i)} I_j^a$$

$$p_i^A = p_i + \sum_{j \in \mu(i)} p_j^a$$

$$I_j^a = I_j^A - I_j^A S_j (S_j^T I_j^A S_j)^{-1} S_j^T I_j^A$$

$$p_j^a = p_j^A + I_j^a c_j + I_j^A S_j (S_j^T I_j^A S_j)^{-1} (\tau_j - S_j^T p_j^A)$$

- ▶ Calculate the accelerations.

$$\ddot{q}_i = (S_i^T I_i^A S_i)^{-1} (\tau_i - S_i^T I_i^A (a_{\lambda(i)} + c_i) - S_i^T p_i^A)$$

$$a_i = a_{\lambda(i)} + c_i + S_i \ddot{q}_i \quad (a_0 = -a_g)$$

Forward Dynamics | Articulated-Body Algorithm

- Previous equations contain a number of common subexpressions, which can be eliminated by defining the following intermediate results:

$$U_i = I_i^A S_i$$

$$D_i = S_i^T U_i$$

$$u_i = \tau_i - S_i^T p_i^A$$

$$a'_i = a_{\lambda(i)} + c_i$$

- In terms of these quantities, Eqs. above simplify to

$$I_j^a = I_j^A - U_j D_j^{-1} U_j^T$$

$$p_j^a = p_j^A + I_j^a c_j + U_j D_j^{-1} u_j$$

$$\ddot{q}_i = D_i^{-1} (u_i - U_i^T a'_i)$$

$$a_i = a'_i + S_i \ddot{q}_i \quad (a_0 = -a_g)$$

Forward Dynamics | Articulated-Body Algorithm

Algorithm:

```
 $v_0 = \mathbf{0}$ 
for  $i = 1$  to  $N_B$  do
   $[\mathbf{X}_J, \mathbf{S}_i, \mathbf{v}_J, \mathbf{c}_J] = \text{jcalc}(\text{jtype}(i), \mathbf{q}_i, \dot{\mathbf{q}}_i)$ 
   ${}^i\mathbf{X}_{\lambda(i)} = \mathbf{X}_J \mathbf{X}_T(i)$ 
  if  $\lambda(i) \neq 0$  then
     ${}^i\mathbf{X}_0 = {}^i\mathbf{X}_{\lambda(i)} {}^{\lambda(i)}\mathbf{X}_0$ 
  end
   $\mathbf{v}_i = {}^i\mathbf{X}_{\lambda(i)} \mathbf{v}_{\lambda(i)} + \mathbf{v}_J$ 
   $\mathbf{c}_i = \mathbf{c}_J + \mathbf{v}_i \times \mathbf{v}_J$ 
   $\mathbf{I}_i^A = \mathbf{I}_i$ 
   $\mathbf{p}_i^A = \mathbf{v}_i \times^* \mathbf{I}_i \mathbf{v}_i - {}^i\mathbf{X}_0^* \mathbf{f}_i^x$ 
end
for  $i = N_B$  to 1 do
   $\mathbf{U}_i = \mathbf{I}_i^A \mathbf{S}_i$ 
   $\mathbf{D}_i = \mathbf{S}_i^T \mathbf{U}_i$ 
   $\mathbf{u}_i = \boldsymbol{\tau}_i - \mathbf{S}_i^T \mathbf{p}_i^A$ 
  if  $\lambda(i) \neq 0$  then
     $\mathbf{I}^a = \mathbf{I}_i^A - \mathbf{U}_i \mathbf{D}_i^{-1} \mathbf{U}_i^T$ 
     $\mathbf{p}^a = \mathbf{p}_i^A + \mathbf{I}^a \mathbf{c}_i + \mathbf{U}_i \mathbf{D}_i^{-1} \mathbf{u}_i$ 
     $\mathbf{I}_{\lambda(i)}^A = \mathbf{I}_{\lambda(i)}^A + {}^{\lambda(i)}\mathbf{X}_i^* \mathbf{I}^a {}^i\mathbf{X}_{\lambda(i)}$ 
     $\mathbf{p}_{\lambda(i)}^A = \mathbf{p}_{\lambda(i)}^A + {}^{\lambda(i)}\mathbf{X}_i^* \mathbf{p}^a$ 
  end
end
 $\mathbf{a}_0 = -\mathbf{a}_g$ 
for  $i = 1$  to  $N_B$  do
   $\mathbf{a}' = {}^i\mathbf{X}_{\lambda(i)} \mathbf{a}_{\lambda(i)} + \mathbf{c}_i$ 
   $\ddot{\mathbf{q}}_i = \mathbf{D}_i^{-1} (\mathbf{u}_i - \mathbf{U}_i^T \mathbf{a}')$ 
   $\mathbf{a}_i = \mathbf{a}' + \mathbf{S}_i \ddot{\mathbf{q}}_i$ 
end
```

pin,aba(model, data, q, v, tau)

- Computes joint accelerations using the Articulated-Body Algorithm (ABA). Used for simulation, control, and real-time dynamic prediction.

Forward Dynamics | Assertion

forward_dynamics_aba.py

```
data = model.createData()

q = np.array([0.5, -0.3])
v = np.array([0.1, 0.2])
tau=np.array([-24.85364489  -8.61740833])

ddotq = pin.aba(model, data, q, v, tau)
```

```
(pin) home@DESKTOP-T4D06E6:~/pinocchio/examples$ python
forward_dynamics_aba.py
nq = 2 nv = 2
Joint accelerations: [-3.76557097e-09  1.000000001e+00]
```


Forward Dynamics | Example from Lecture 2

Updating pose over time with ABA and integration

```
def sim_loop():
    tau0 = np.zeros(model.nv)
    qs = [q1]
    vs = [v0]
    nsteps = 100
    for i in range(nsteps):
        q = qs[i]
        v = vs[i]
        a1 = pin.aba(model, data, q, v, tau0)
        vnext = v + dt * a1
        qnext = pin.integrate(model, q, dt * vnext)
        qs.append(qnext)
        vs.append(vnext)
        viz.display(qnext)
        viz.drawFrameVelocities(frame_id=frame_id)
    return qs, vs

qs, vs = sim_loop()
```

- Inverse Dynamics (RNEA): Computes joint torques from desired accelerations using a recursive Newton–Euler formulation.
- Forward Dynamics (ABA): Computes joint accelerations from torques through articulated–body inertia propagation.
 - Articulated Body treats each subtree as a single equivalent rigid body with combined inertia and bias forces.
- Pinocchio validation: Derived formulas and algorithms were verified numerically using `pin.rnea()` and `pin.aba()`.

Reference

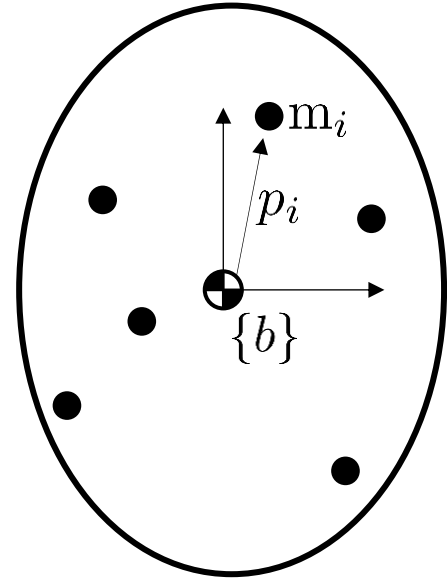
► Base Velocity and COM Constraint

$$\sum_i m_i p_i = 0 \quad \nu_b = \begin{bmatrix} \omega_b \\ v_b \end{bmatrix}$$

► Linear velocity and acceleration

$$\dot{p}_i = v_b + \omega_b \times p_i$$

$$\begin{aligned} \ddot{p}_i &= \dot{v}_b + \frac{d}{dt} \omega_b \times p_i + \omega_b \times \frac{d}{dt} p_i \\ &= \dot{v}_b + \dot{\omega}_b \times p_i + \omega_b \times (v_b + \omega_b \times p_i) \\ \ddot{p}_i &= \dot{v}_b + [\dot{\omega}_b] p_i + [\omega_b] v_b + [\omega_b]^2 r_i \end{aligned}$$



Reference

► Dynamics of a Single Rigid Body

$$\begin{aligned} f_i &= m_i \ddot{p}_i \\ &= m_i (\dot{v}_b + [\dot{\omega}_b] p_i + [\omega_b] v_b + [\omega_b]^2 p_i) \end{aligned}$$

$$m_i = [p_i] f_i$$

$$\begin{aligned} f_b &= \sum_i m_i (\dot{v}_b + [\dot{\omega}_b] p_i + [\omega_b] v_b + [\omega_b]^2 p_i) \\ &= \sum_i m_i (\dot{v}_b + [\omega_b] v_b) - \sum_i m_i [p_i] \dot{\omega}_b + \sum_i m_i [p_i] [\omega_b] \omega_b \\ &= \sum_i m_i (\dot{v}_b + [\omega_b] v_b) \\ &= m (\dot{v}_b + [\omega_b] v_b) \end{aligned}$$

Reference

► Dynamics of a Single Rigid Body

$$\begin{aligned}m_b &= \sum_i m_i [p_i] (\dot{v}_b + [\dot{\omega}_b] p_i + [\omega_b] v_b + [\omega_b]^2 p_i) \\&= \sum_i m_i [p_i] \dot{v}_b + \sum_i m_i [p_i] [\omega_b] v_b + \sum_i m_i [p_i] ([\dot{\omega}_b] p_i + [\omega_b]^2 p_i) \\&= \sum_i m_i (-[p_i]^2 \dot{\omega}_b - [p_i]^T [\omega_b]^T [p_i] \omega_b) \\&= \sum_i m_i (-[p_i]^2 \dot{\omega}_b - [\omega_b] [p_i]^2 \omega_b) \\&= (-\sum_i m_i [p_i]^2) \dot{\omega}_b + [\omega_b] (-\sum_i m_i [p_i]^2) \omega_b \\&= \mathcal{I}_b \dot{\omega}_b + [\omega_b] \mathcal{I}_b \omega_b \\ \mathcal{F}_b &= \begin{bmatrix} m_b \\ f_b \end{bmatrix} = \begin{bmatrix} \sum_i m_i \\ \sum_i f_i \end{bmatrix} = \begin{bmatrix} \mathcal{I}_b \dot{\omega}_b + [\omega_b] \mathcal{I}_b \omega_b \\ m(\dot{v}_b + [\omega_b] v_b) \end{bmatrix}\end{aligned}$$

Reference

► Moment of inertia and inertia tensor

$$\mathcal{I}_b = - \sum_i m_i [p_i]^2 \in \mathbb{R}^{3 \times 3}$$

$$\begin{aligned} \mathcal{I}_b &= \begin{bmatrix} \mathcal{I}_{xx} & \mathcal{I}_{xy} & \mathcal{I}_{xz} \\ \mathcal{I}_{xy} & \mathcal{I}_{yy} & \mathcal{I}_{yz} \\ \mathcal{I}_{xz} & \mathcal{I}_{yz} & \mathcal{I}_{zz} \end{bmatrix} \\ &= \begin{bmatrix} \sum m_i (y_i^2 + z_i^2) & - \sum m_i x_i y_i & - \sum m_i x_i z_i \\ - \sum m_i x_i y_i & \sum m_i (x_i^2 + z_i^2) & - \sum m_i y_i z_i \\ - \sum m_i x_i z_i & - \sum m_i y_i z_i & \sum m_i (x_i^2 + y_i^2) \end{bmatrix} \end{aligned}$$

$$\mathcal{I}_{xx} = \int_{\mathcal{B}} (y^2 + z^2) \rho(x, y, z) dV$$

$$\mathcal{I}_{xy} = - \int_{\mathcal{B}} xy \rho(x, y, z) dV$$

$$\mathcal{I}_{yy} = \int_{\mathcal{B}} (x^2 + z^2) \rho(x, y, z) dV$$

$$\mathcal{I}_{xz} = - \int_{\mathcal{B}} xz \rho(x, y, z) dV$$

$$\mathcal{I}_{zz} = \int_{\mathcal{B}} (x^2 + y^2) \rho(x, y, z) dV$$

$$\mathcal{I}_{yz} = - \int_{\mathcal{B}} yz \rho(x, y, z) dV$$

Reference

'GitHub – stack-of-tasks/pinocchio: A fast and flexible implementation of Rigid Body Dynamics algorithms and their analytical derivatives'. github.com/stack-of-tasks/pinocchio.

Lynch, K. M., & Park, F. C. (2017). *Modern robotics*. Cambridge University Press.

Featherstone, R. (2008). *Rigid body dynamics algorithms*. Boston, MA: Springer US.

03

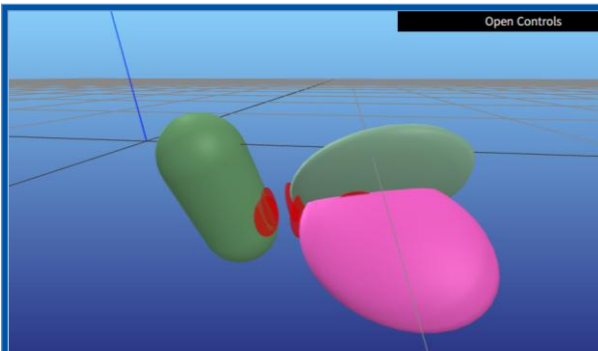
Dynamics (2)

Learning Objectives

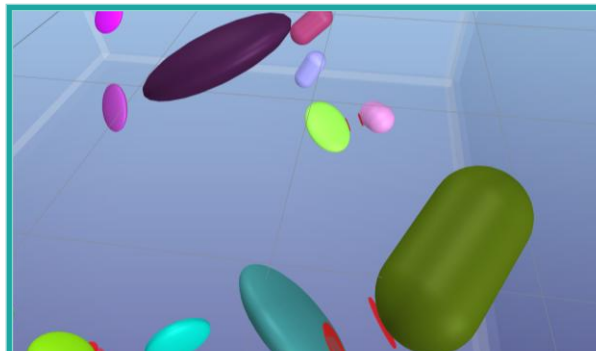
By the end of this lecture, you should be able to:

- Collision detection
- Contact modeling
- Dynamics integration
- Basic control (PD control and Joint trajectory)

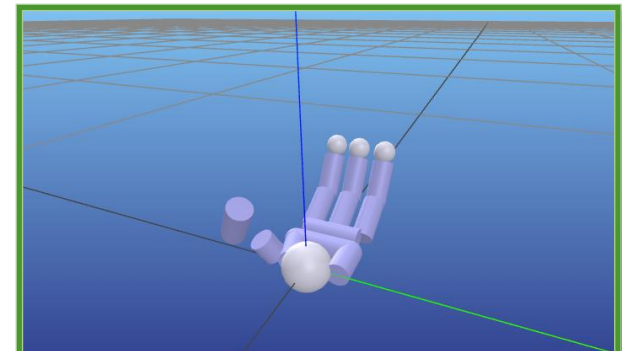
through “4-simu.ipynb” from [supaero2025](#)



Collision detection



Contact modeling

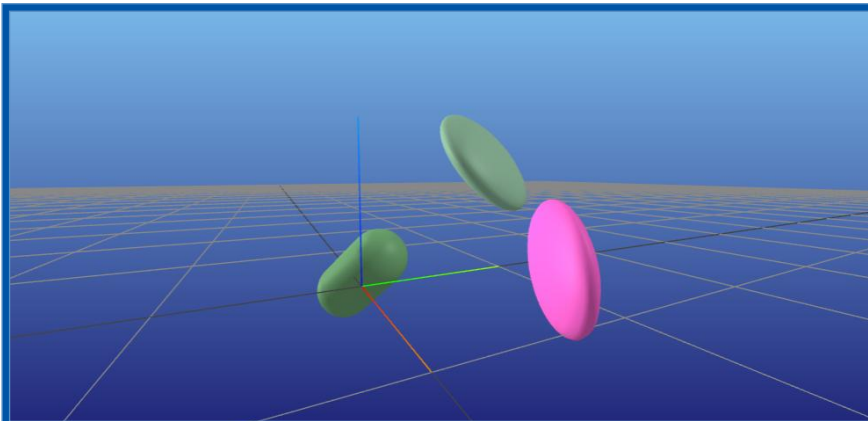


Basic control

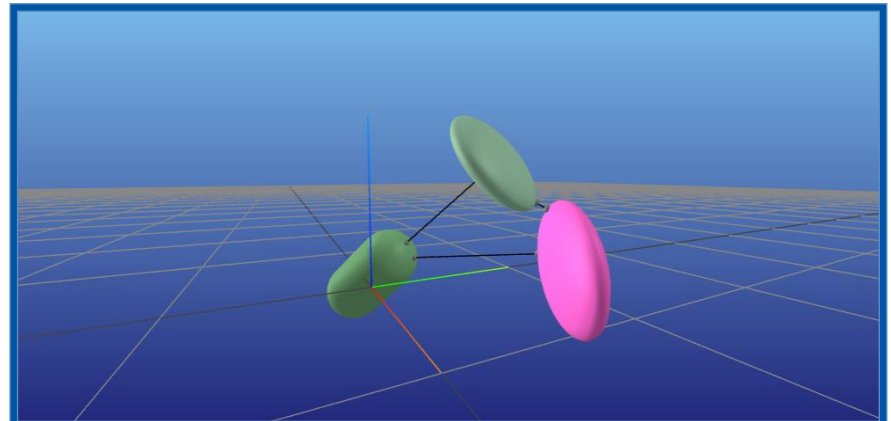
Dynamics | Collision Detection

► Basic example of Collision Detection

```
model, geom_model = buildSceneThreeBodies()  
data = model.createData()  
geom_data = geom_model.createData()  
  
viz = MeshcatVisualizer(model=model, collision_model=geom_model,  
                        visual_model=geom_model)  
  
q=pin.randomConfiguration(model)  
viz.display(q)
```



Scene Built using meshcat visualizer



Distances between models displayed

Dynamics | Collision Detection

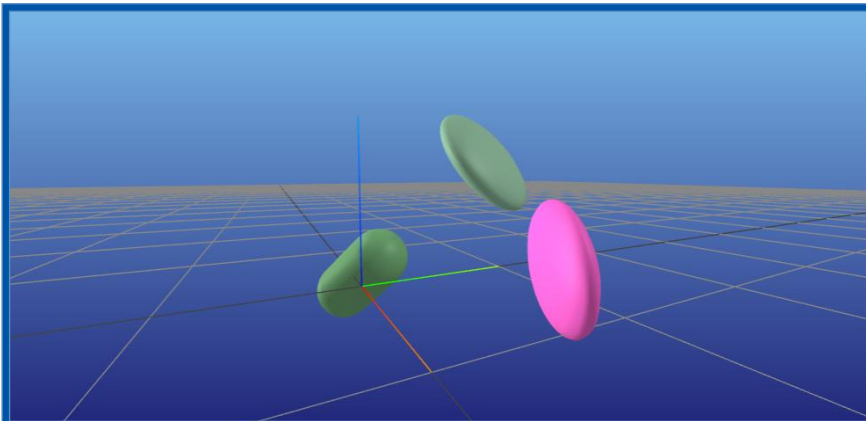
► Basic example of Collision Detection

```
mcWitnesses = DisplayCollisionWitnessesInMeshcat(viz)

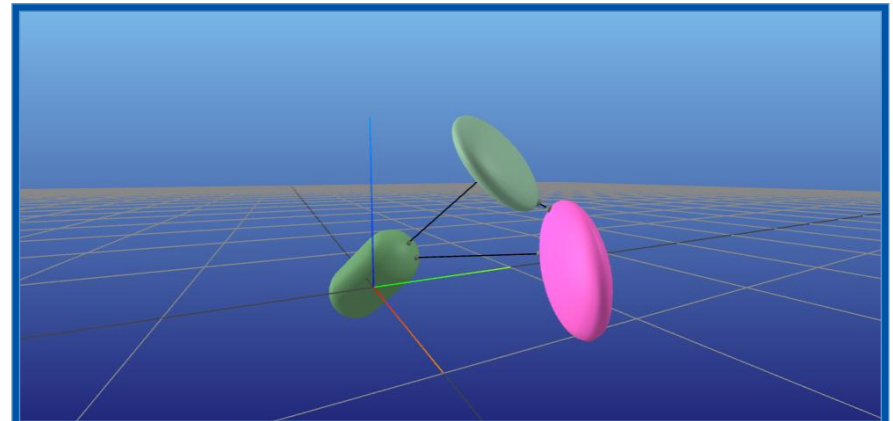
pin.computeDistances(model,data,geom_model,geom_data,q)
mcWitnesses.displayDistances(geom_data)
```

computeDistances returns signed distance and closest points

computeCollisions returns contact points when penetration occurs



Scene Built using meshcat visualizer



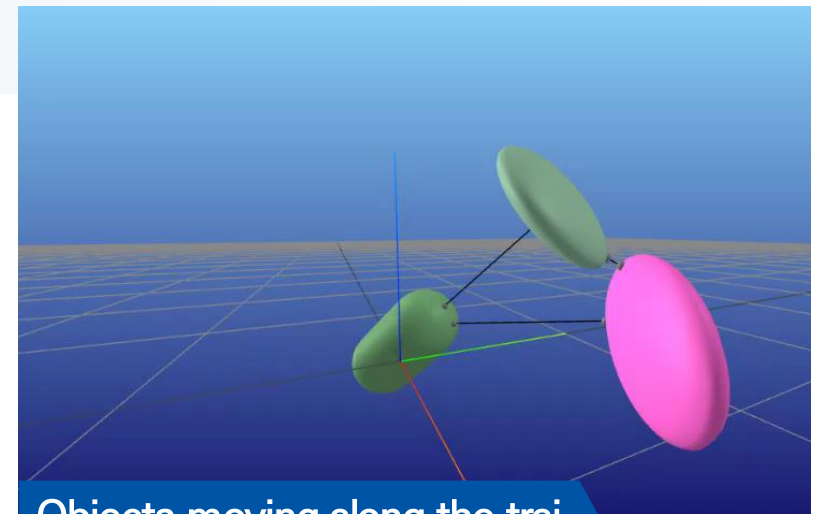
Distances between models displayed

Dynamics | Collision Detection

- Update the robot position along an arbitrary trajectory

```
v = (np.random.rand(model.nv)*2-1)*1e-3
r0 = [ np.linalg.norm(q[7*i:7*i+3]) for i in range(model.nq//7) ]
for t in range(100):
    q = pin.integrate(model,q,v*10)
    for i in range(model.nq//7):
        q[7*i:7*i+3] *= r0[i]/np.linalg.norm(q[7*i:7*i+3])
    viz.display(q)

    pin.computeDistances(model,data,geom_model,geom_data,q)
    mcWitnesses.displayDistances(geom_data)
    time.sleep(.01)
```



For each geometry pair we get:

- Two closest points p_1, p_2 on the surfaces
- A unit normal n pointing from body 1 to body 2
- Signed distance d (projection of $p_2 - p_1$ on the normal)

$$d = n^T(p_2 - p_1)$$

Each components can be called by:

```
d = geom_data.distanceResults[0]
print("p1:", d.getNearestPoint1())
print("p2:", d.getNearestPoint2())
print("n:", d.normal)
print("dist:", d.min_distance)
```

■ Contact frames are defined at the witness points satisfying:

- Origin at p_1 or p_2 , which is the nearest point from the contact
- z-axis aligned with normal n
- Implemented using Pinocchio SE3

■ Use of the defined wrapper

```
# Option 1
pin.computeCollisions(model,data,geom_model,geom_data,q,False)
contact_models = createContactModelsFromCollisions(model,data,geom_model,geom_data)
contact_datas = [ cm.createData() for cm in contact_models ]

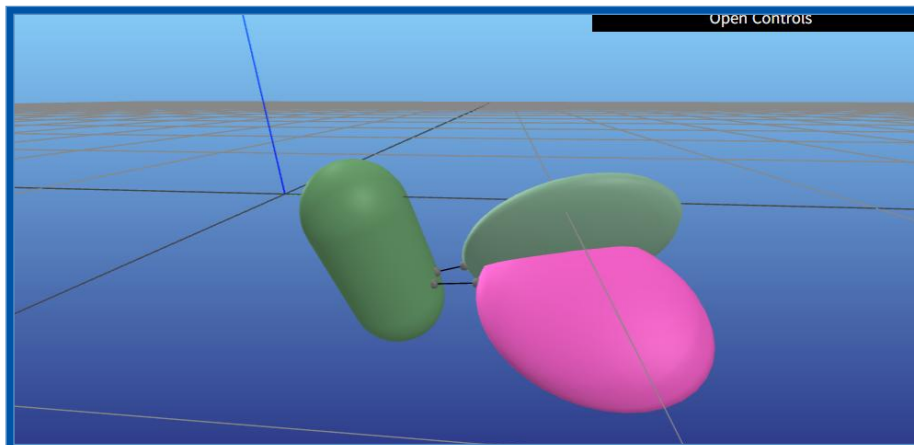
# Option 2
pin.computeDistances(model,data,geom_model,geom_data,q)
contact_models = createContactModelsFromDistances(model,data,geom_model,geom_data,
                                                    threshold=10) # threshold in meter
contact_datas = [ cm.createData() for cm in contact_models ]
```

Dynamics | Contact Models

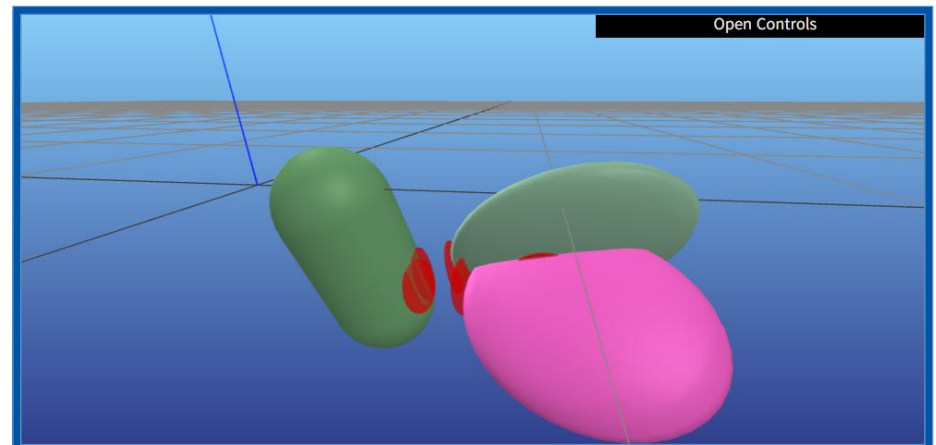
Contact frames are defined at the witness points satisfying:

- Origin at p_1 or p_2 , which is the nearest point from the contact
- z-axis aligned with normal n
- Implemented using Pinocchio SE3

Displaying the contact models



Distances between models displayed



Contact models displayed as red patches

Dynamics | Building a simulator

Building a simple kinematic simulator

- ▶ Consider only geometry and configuration, ignore dynamics
- ▶ Goal: remove penetration by directly adjusting the configuration
- ▶ For each iteration:
 - ▶ Detect collisions and compute signed distance
 - ▶ Modify the current configuration velocity with the `PUSH_FACTOR`
 - ▶ Display the current configuration (`updateVisualObjects`)

Use of each Hyper parameter

PUSH_FACTOR Defines the magnitude of the displacement applied to colliding bodies

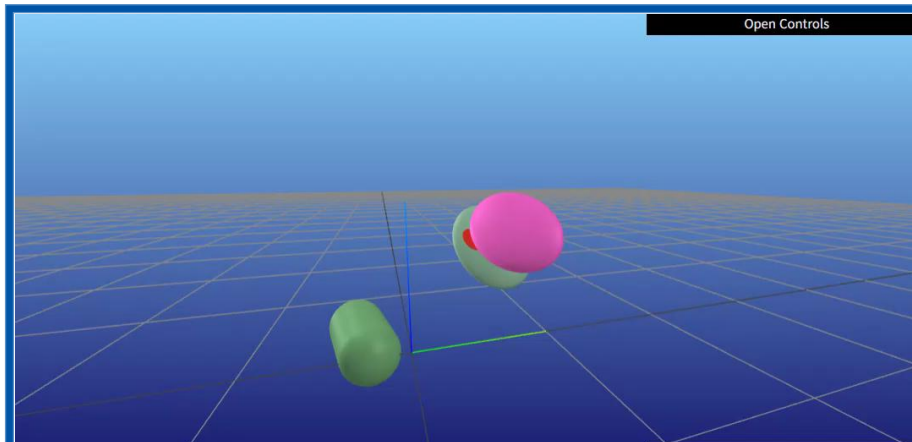
EPSILON Sets the minimum distance threshold to avoid body interpenetration

NB_ITER Specifies the number of iterations for resolving collisions

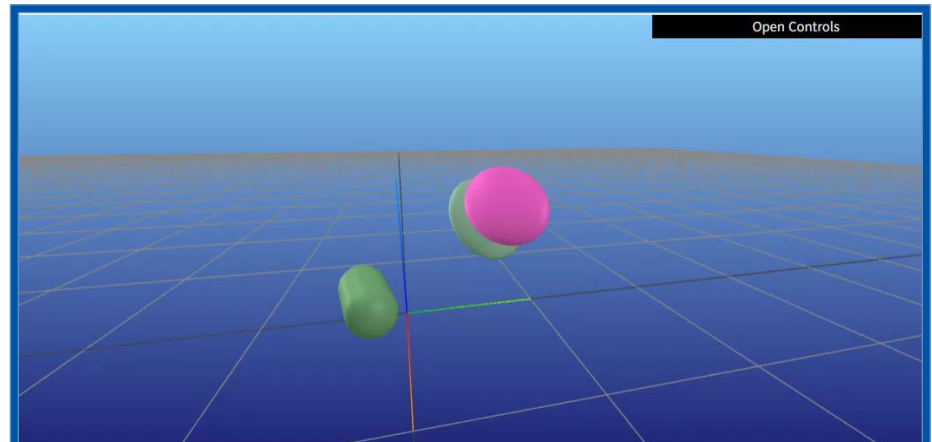
USE_DISTANCE Determines whether to compute contact information based on collision detection or distance queries

Dynamics | Building a simulator

- For each iteration:
 - Compute distances or collisions
 - Build contact models from results
 - For each contact:
 - compute signed distance
 - compute a push velocity or increment for the bodies
 - Integrate configuration with Pinocchio integrate



PUSH_FACTOR = .1

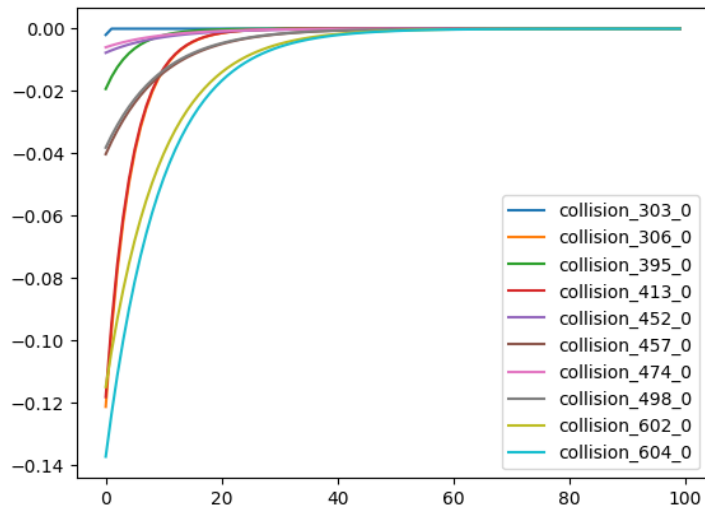
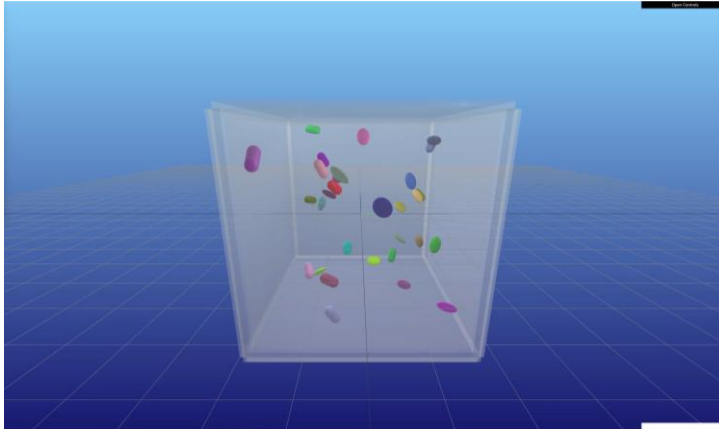


PUSH_FACTOR = 10

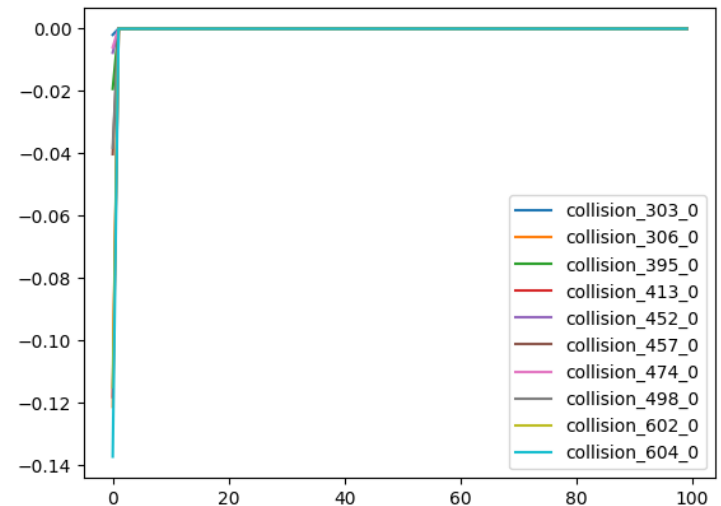
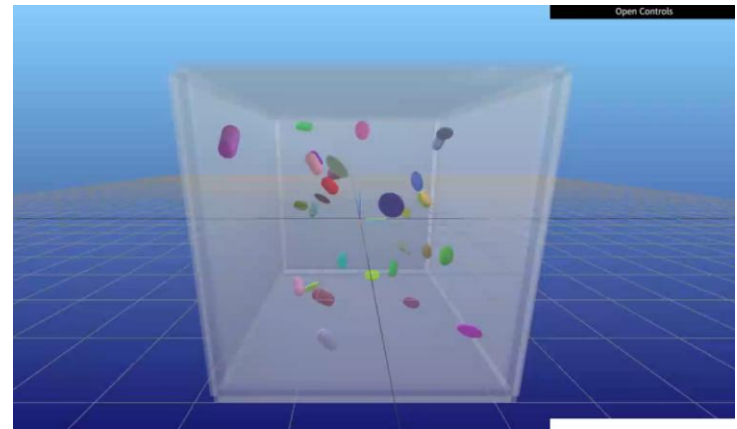
Dynamics | Building a simulator

With a more complex scene

PUSH_FACTOR = 0.1



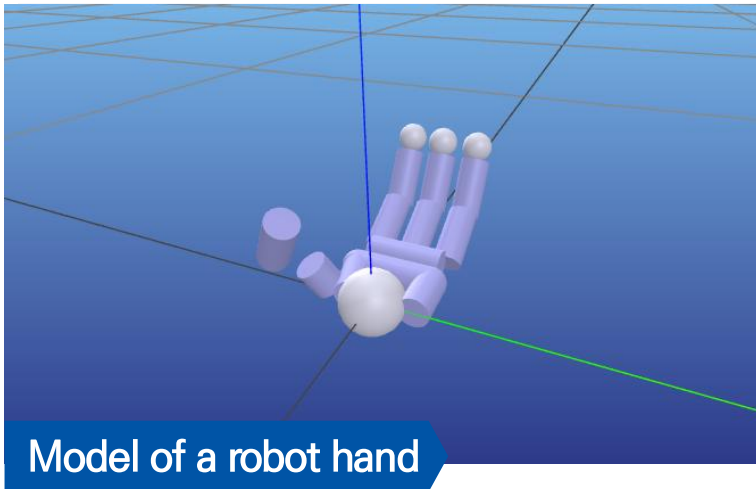
PUSH_FACTOR = 3



Contact-less simulator

Simulate the robot hand's motion by computing joint accelerations using the mass matrix (CRBA) and nonlinear effects (NLE), without considering contact forces

- ▶ PD Control: Introduce proportional-derivative (PD) control to track desired joint trajectories by adjusting joint torques based on position and velocity errors.
- ▶ Forward dynamics: Use the computed mass matrix and drift term to find accelerations, then integrate joint velocities and positions over time.



```
model, geom_model = buildSceneRobotHand()  
q0 = model.referenceConfigurations['default']  
data = model.createData()
```

Evaluating the core elements of the dynamics

$$\text{ID : } M(q)a_q + b(q, v_q) = \tau_q$$

$$\text{FD : } a_q = M^{-1}(q)(\tau_q - b(q, v_q))$$

$M(q)$ joint-space mass matrix via Composite Rigid Body Algorithm(CRBA)

$b(q, v_q)$ Nonlinear effects (Coriolis, Centrifugal, Gravity) via NLE

```
print(pin.aba(model, data, q, vq, tauq)-aq)
print(pin.rnea(model, data, q, vq, aq)-tauq)
print(pin.rnea(model, data, q, vq, vq*0)-pin.nle(model, data, q, vq))
```

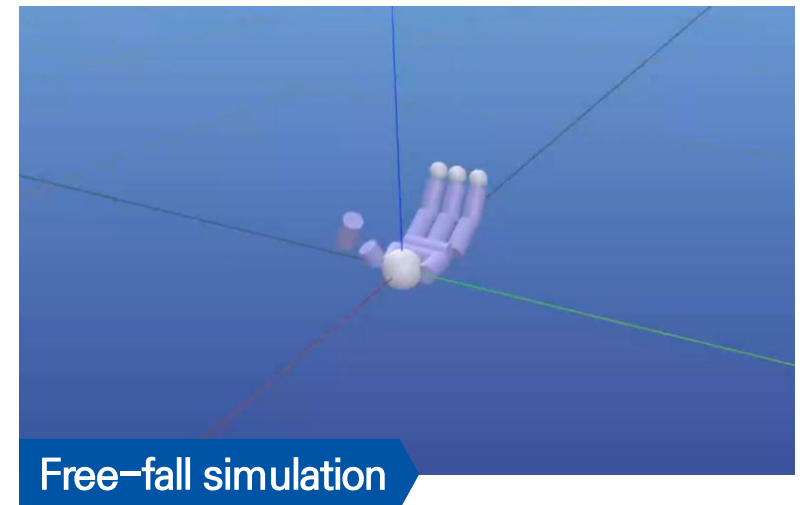
```
[ 5.55111512e-17 -6.93889390e-17 -6.66133815e-16  8.88178420e-16
 8.88178420e-16 -4.44089210e-16  2.66453526e-15 -5.32907052e-15
 3.33066907e-16 -4.44089210e-16  0.00000000e+00  1.66533454e-16
 0.00000000e+00  4.44089210e-16]
[ 1.33226763e-15  3.88578059e-16 -6.66133815e-16 -6.66133815e-16
-1.11022302e-16  4.99600361e-16  0.00000000e+00  0.00000000e+00
 5.55111512e-17  0.00000000e+00  2.22044605e-16 -1.11022302e-16
 2.22044605e-16 -5.55111512e-17]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

$\approx$ [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

$\approx$ [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

Free-falling dynamics

- ▶ Set $\tau_q = 0$
- ▶ Gravity is included in $b(q, v_q)$
- ▶ Loop over time:
 - ▶ Compute M, b
 - ▶ Compute a_q using the equation of motion or pinocchio's function:
FD : $a_q = M^{-1}(q)(\tau_q - b(q, v_q))$
`aq = pin.aba(model, data, q, vq, tauq)`
 - ▶ Integrate v_q, q
 - ▶ Visualize robot hand motion



Proportional-derivative and computed torque

► Set desired trajectory q_{des} via TrajRef (builds sinusoidal trajectories on each joint)

► Loop over time:

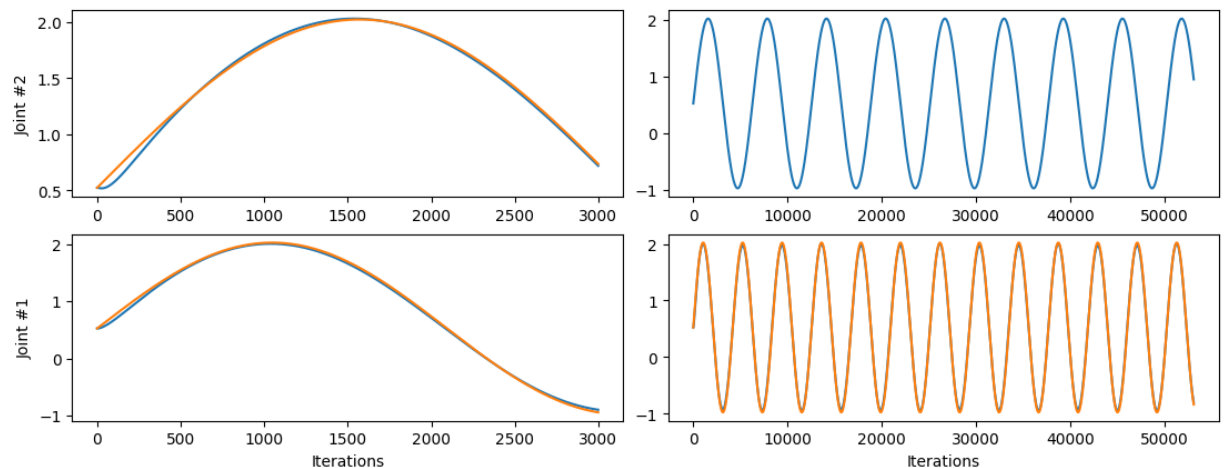
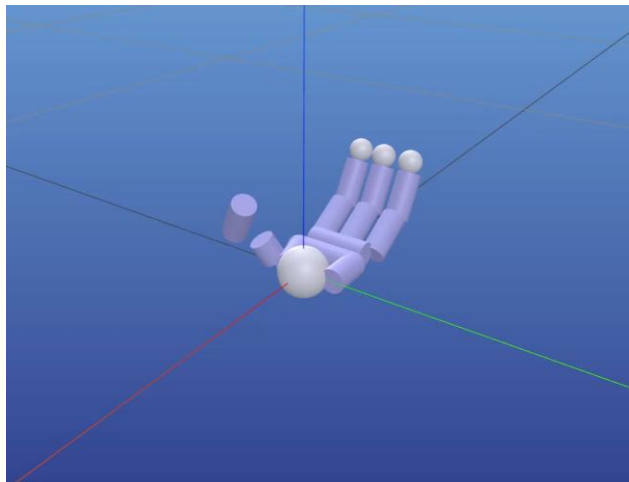
► Compute the PD control:

$$\tau_q = -K_p(q - q_{des}) - K_v(v_q - \dot{q}_{des}) + \ddot{q}_{des} \quad (K_v = 2\sqrt{K_p})$$

► Compute a_q using the equation of motion or pinocchio's function:

► Integrate v_q, q

► Visualize robot hand motion



Friction-less simulation

- ▶ The jacobian maps the configuration velocity to the relative velocity of contact points
- ▶ `getConstraintsJacobian` returns full contact Jacobian
- ▶ For frictionless contact:
 - ▶ Consider the normal component only
 - ▶ Select the last row and flip sign to make normal component positive

```
contact_model = pin.RigidConstraintModel(pin.ContactType.CONTACT_3D,model,1,
                                         pin.SE3.Identity(), 0,
                                         pin.SE3.Identity(), pin.LOCAL)
contact_data = contact_model.createData()
pin.computeJointJacobians(model,data,q0)
pin.getConstraintsJacobian(model,data,[contact_model],[contact_data])
Jc = -pin.getConstraintsJacobian(model,data,[contact_model],[contact_data])[2::3]
```

```
array([[ -8.32667268e-17,  4.16333634e-17, -1.00000000e+00,
        -0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
        -0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
        -0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
        -0.00000000e+00, -0.00000000e+00, -0.00000000e+00,
        -0.00000000e+00, -0.00000000e+00, -0.00000000e+00]])
```

► If no collision:

- Compute free velocity from ABA:

$$v_f = v + \Delta t a_{free}$$

► If collisions:

- Choose v_+ as close as possible to free-fall velocity v_f
- With contact forces enforcing non-penetration

$$\min_{v_+} f(v_+) = \frac{1}{2} \|v_+ - v_f\|_M^2 == \frac{1}{2} (v_+ - v_f)^T M (v_+ - v_f)$$

- Satisfying following inequality:

$$J_c v_+ \geq 0$$

Friction-less simulation

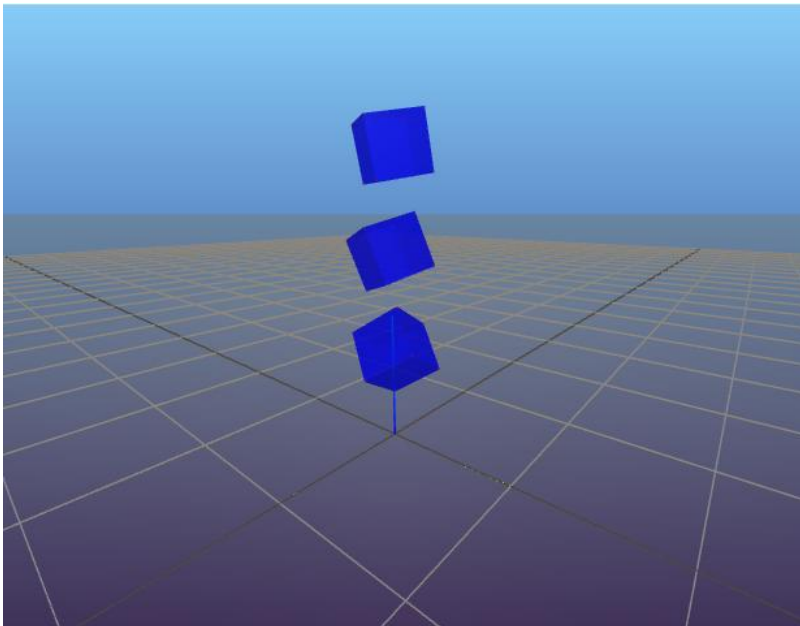
➤ Simulation loop:

- Check the collisions, create the contact model and the corresponding contact data
- Evaluate the free-fall next velocity v_f with ABA
- If no collision, then the next velocity is v_f , continue.
- If collisions, then evaluate the inertia matrix M with `pin.crba`, and the Jacobian J with `pin.getConstraintJacobian`; form the primal QP, solve it. The result is the next velocity, the Lagrange multipliers are the contact forces.
- Update Visualizer and log.

Friction-less simulation

► Simulation loop:

- Check the collisions, create the contact model and the corresponding contact data
- Evaluate the free-fall next velocity v_f with ABA
- Compute the next velocity v_f considering the collisions
- Update Visualizer and log.



05

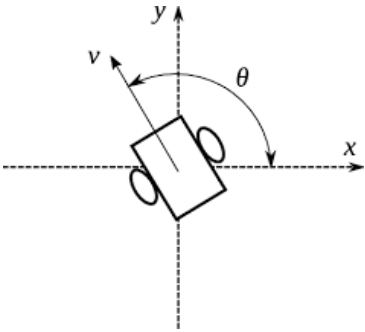
Optimal Control

Learning Objectives

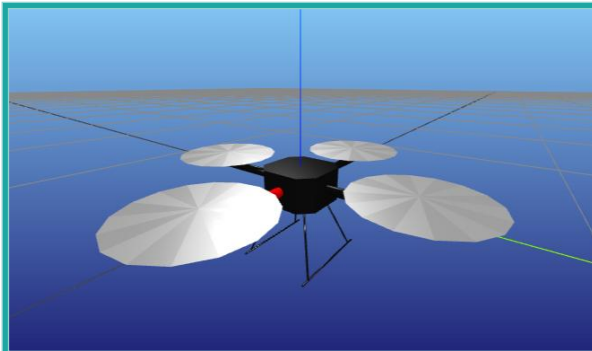
Optimize a cost function defined over the system states and control inputs of a dynamical system

- Three-stage pipeline of Crocoddyl illustrated with the unicycle model
- Bicopter model implementation
- Trajectory optimization for the Panda manipulator with single and multiple targets

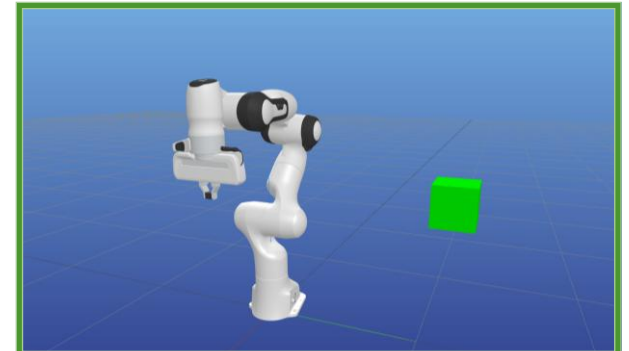
through “5-1_controlopt_intro.ipynb” and “5-2_controlopt_multibody.ipynb” from [supaero2025](#)



Unicycle



Bicopter



Panda Manipulator

Three-stage pipeline of Crocoddyl

► Action model (model definition)

Defines the system dynamics $x_{next} = f(x, u)$ and the cost function.

In the example of unicycle, we use the built-in `crocoddyl.ActionModelUnicycle()`

► Shooting Problem (problem formulation)

Prepares the simulation from the initial state x_0 over a time horizon T

Defines as `crocoddyl.ShootingProblem(x0, [runningmodel]*T, terminalmodel)`

► Solve the Optimal Control Problem(OCP)

Through Differential Dynamic Programming(DDP)

`ddp.solve()` returns optimal trajectory and control input

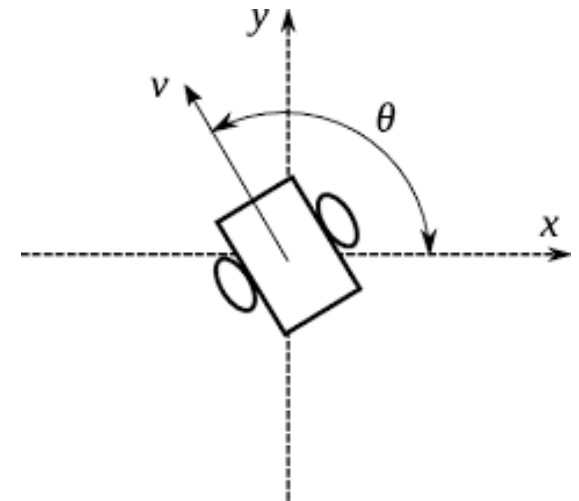
- DDP computes derivatives around the current trajectory and gradually adjusts the control inputs in the direction that reduces cost.

Optimal Control | Unicycle example

System modeling (Dynamics & Cost)

- ▶ The unicycle cannot move sideways; it can only move forward v and rotate ω
- ▶ State $x = [x, y, \theta]$: position and direction
- ▶ Control $u = [v, \omega]$: linear velocity and angular velocity
- ▶ Cost function: drive the robot to the origin while avoiding excessively fast motion

$$\text{Cost} = \frac{1}{2}(\|\omega_x x\|^2 + \|\omega_u u\|^2)$$

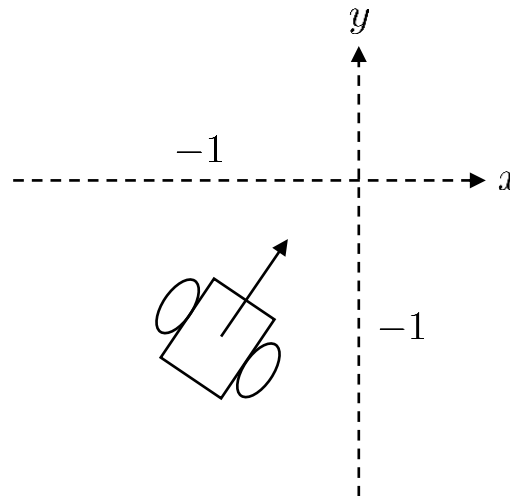
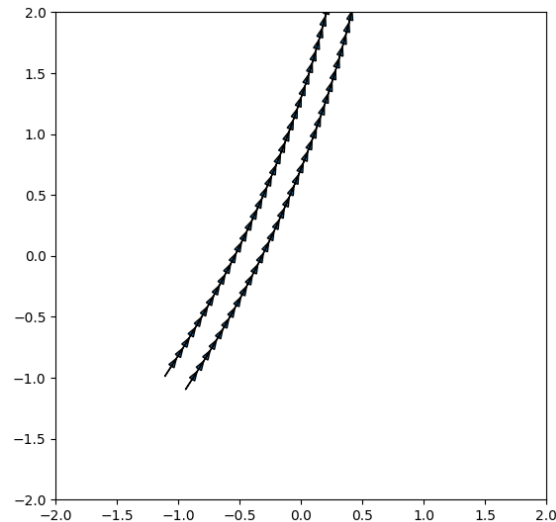


Optimal Control | Unicycle example

- State $x = [x, y, \theta]$: position and direction

With initial condition $[-1, -1, 1]$

The original trajectory looks like this:



- Cost function: drive the robot to the origin while avoiding excessively fast motion

```
model.costWeights = np.array([
    1, # state weight
    1 # control weight
])
```

Optimal Control | Unicycle example

- Cost function: drive the robot to the origin while avoiding excessively fast motion

```
model.costWeights = np.array([  
    1, # state weight  $\omega_x$   
    1  # control weight  $\omega_u$   
])
```

$$\text{Cost} = \frac{1}{2}(\|\omega_x x\|^2 + \|\omega_u u\|^2)$$

- Problem formulation

```
problem = crocoddyl.ShootingProblem(x0, [runningModel] * T, terminalModel)
```

x0 the state from which the robot begins the simulation

[runningModel]*T A list of T identical models applied at each time step. This defines $x_{t+1} = f(x_t, u_t)$ and Cost_t

terminalModel The model applied only at the final time step T

Optimal Control | Unicycle example

► Solve the Optimal Control Problem(OCP)

- DDP computes derivatives around the current trajectory and gradually adjusts the control inputs in the direction that reduces cost.

```
ddp = crocoddyl.SolverDDP(problem)
```

ddp.xs ► Optimized state list $[x_0, x_1, \dots, x_T]$

ddp.us ► Optimized control input list $[u_0, u_1, \dots, u_{T-1}]$

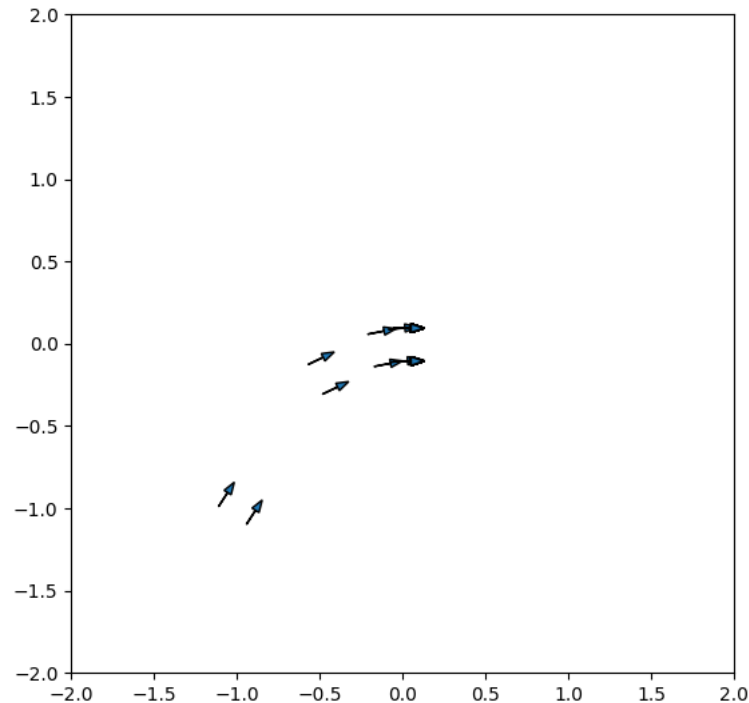
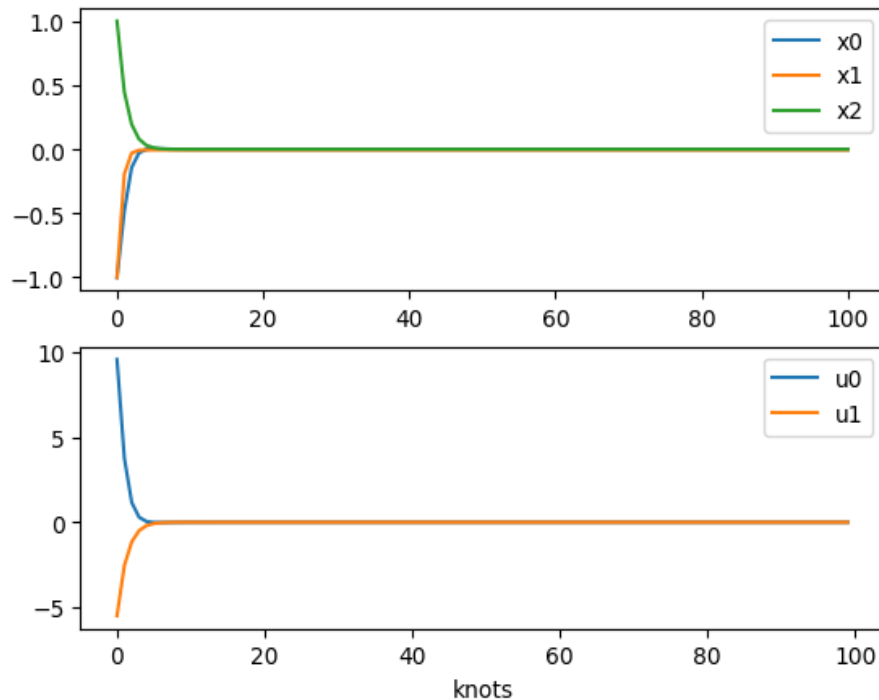
ddp.cost ► The final value of the cost function after optimization is complete

ddp.iter ► The number of iterations required to converge to the optimal solution

Optimal Control | Unicycle example

- ▶ However, the resulting trajectory is not smooth.

To improve smoothness, we need to adjust the state and control weights in the cost function.



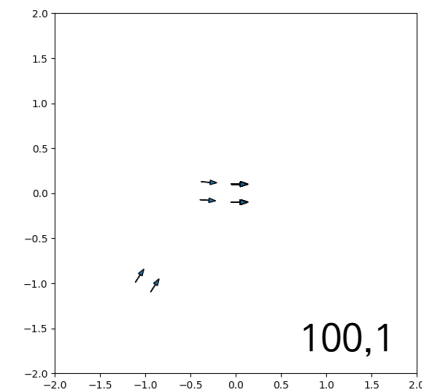
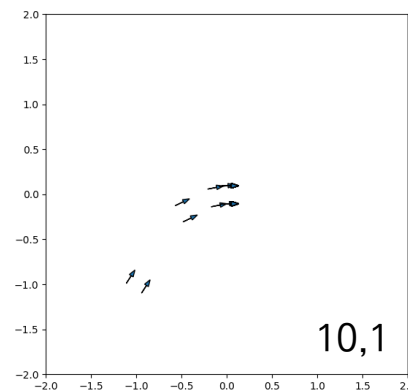
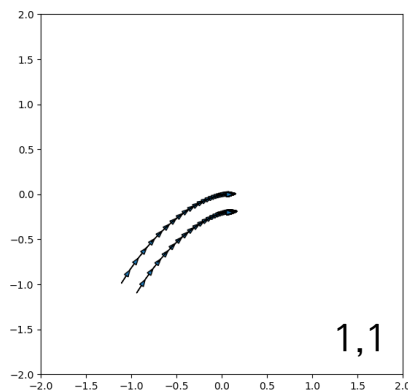
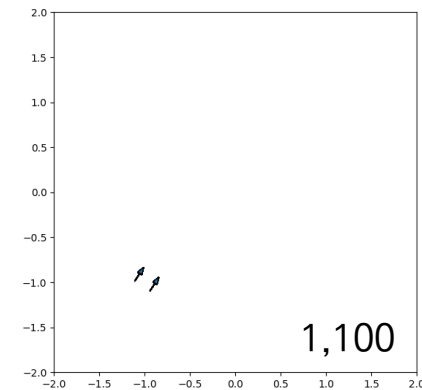
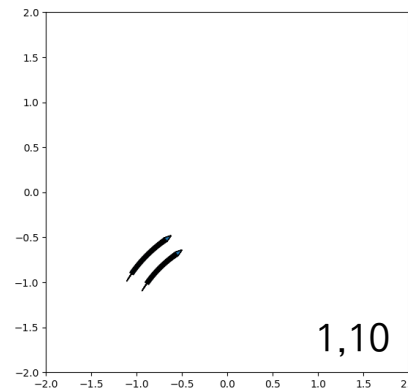
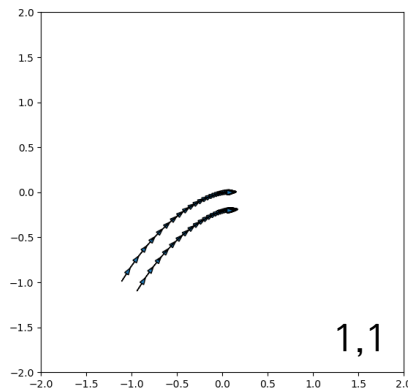
Optimal Control | Unicycle example

► State weight

determines how sharply the cost increases when there is a deviation between the current robot state and the target state.

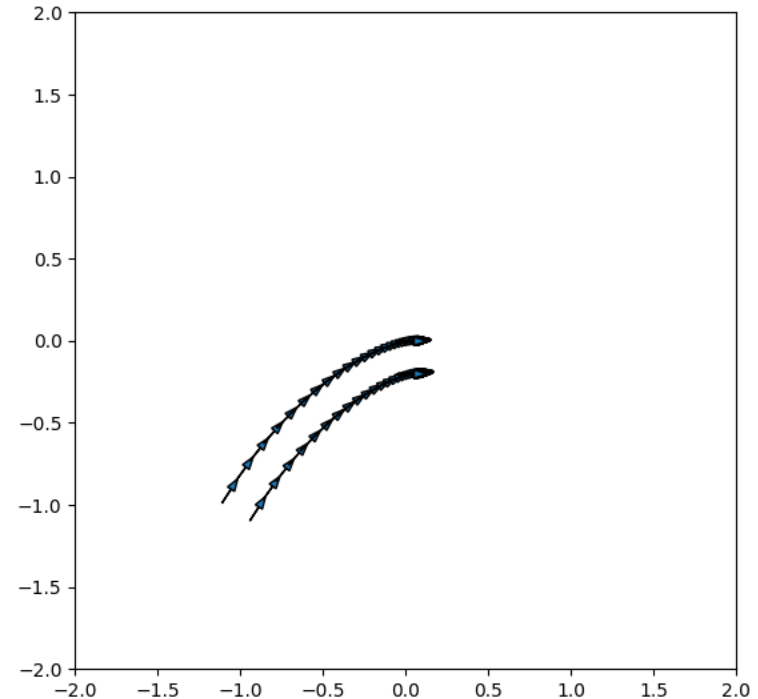
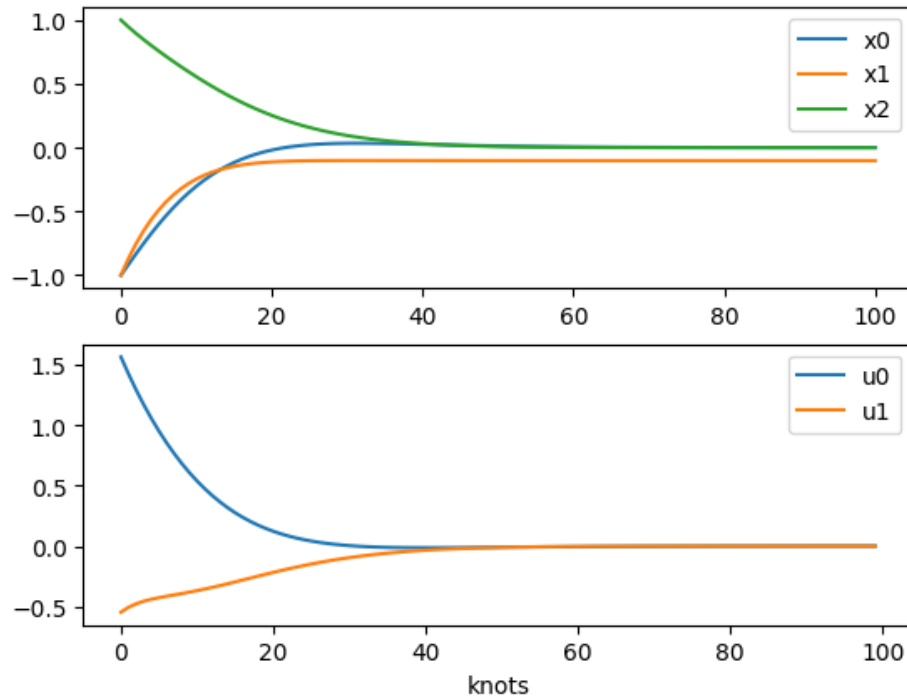
► Control weight

determines how steeply the cost rises as the robot applies larger motor or actuator forces



Optimal Control | Unicycle example

- ▶ Setting both weights to 1 results in a smooth trajectory. Consequently, the robot follows a smooth path toward the origin.

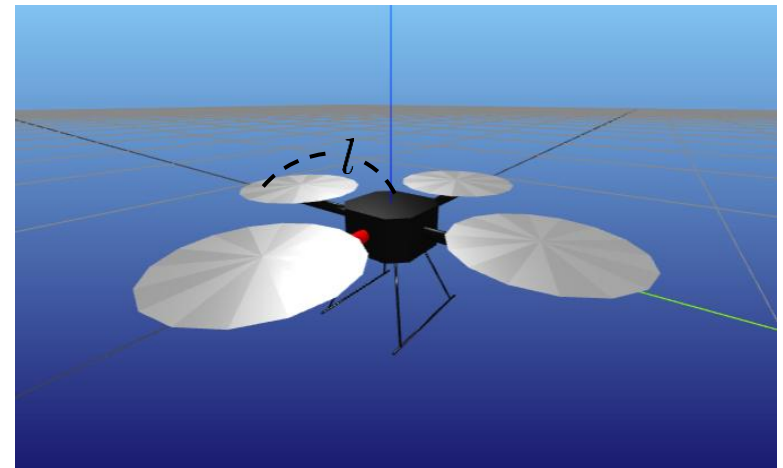


Optimal Control | Bicopter example

System modeling (Dynamics & Cost)

- State ($x \in \mathbb{R}^6$) : $[x, z, \theta, \dot{x}, \dot{z}, \dot{\theta}]$: position and velocity
- Control ($u \in \mathbb{R}^2$) : $[f_r, f_l]$: thrusts of the right and left propellers
- Equation of motion: follows the Newton–Euler formulation
 - The linear and angular forces due to the propellers are easy to express in the local frame:
$$f = (0, f_r + f_l, (f_l - f_r)l)$$
 - The acceleration in world frame is then obtained by rotating the linear forces, dividing by mass and inertia and adding the gravity:

$$a = \left(\frac{f[1] \sin(\theta)}{m}, \frac{f[1] \cos(\theta)}{m} - g, \frac{f[2]}{i} \right)$$



Crocoddyl separates the continuous-time physical model from the discretization (integration) used to compute the next state.

- ▶ DAM (Differential Action Model): defines the differential equation and the instantaneous cost

$$\dot{x} = f(x, u)$$

- ▶ IAM (Integrated Action Model): takes the DAM and integrates it over a time step to obtain the next state. The code uses `IntegratedActionModelEuler`, which applies Euler integration:

$$x_{k+1} = x_k + \dot{x} \Delta t$$

Model definition workflow

- Define the physical equations of motion.
- Wrap them with an automatic differentiation model.
- Apply an integrator to map accelerations to state updates (position and velocity).

```
dam = DifferentialActionModelBicopter
damND = DifferentialActionModelNumDiff(dam, True)
iam = IntegratedActionModelEuler(damND, dt)
```

- Define the terminal model according to the task objective.

Set goals such as precision and stability.

```
terminalDam.costWeights[0] = 10000
terminalDam.costWeights[1] = 10000
terminalDam.costWeights[2] = 1000
```

Match the target position while avoiding abrupt braking.

```
terminalDam.costWeights[0] = 500 # horizontal position
terminalDam.costWeights[1] = 500 # vertical position
...
terminalDam.costWeights[4] = 10 # horizontal velocity
terminalDam.costWeights[5] = 10 # vertical velocity
terminalDam.costWeights[6] = 10 # angular velocity
```

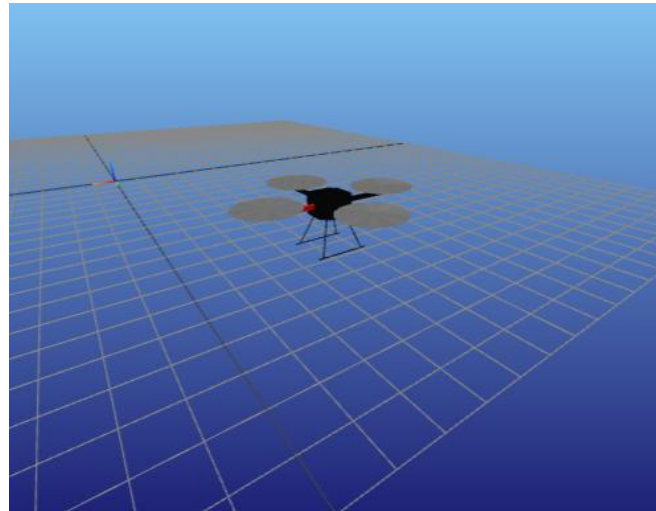
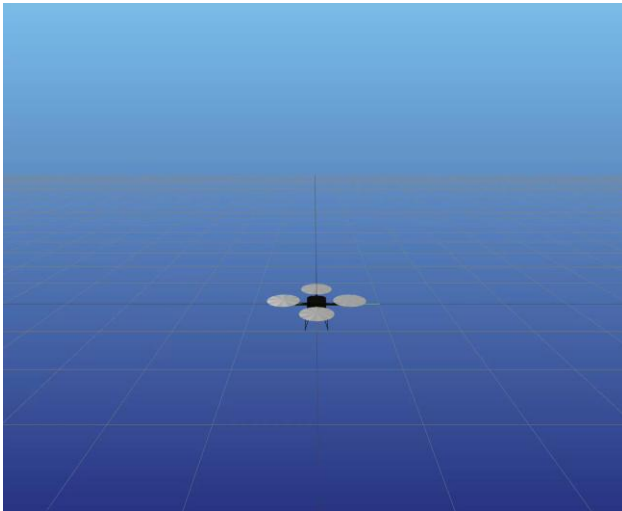
Optimal Control | Bicopter example

- Define the optimal control problem and solve it

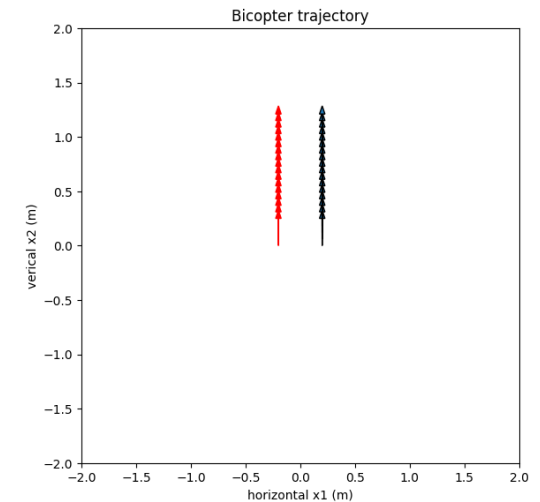
```
problem = crocoddyl.ShootingProblem(x0, [iam] * T, terminalIam)

ddp = crocoddyl.SolverDDP(problem)
ddp.setCallbacks([crocoddyl.CallbackLogger(), crocoddyl.CallbackVerbose()])

done = ddp.solve([], [], 300)
assert(done)
```



```
x0 = np.array([0.0, 0.0, 0.0, 0.0, 2.0, 0.0])
```

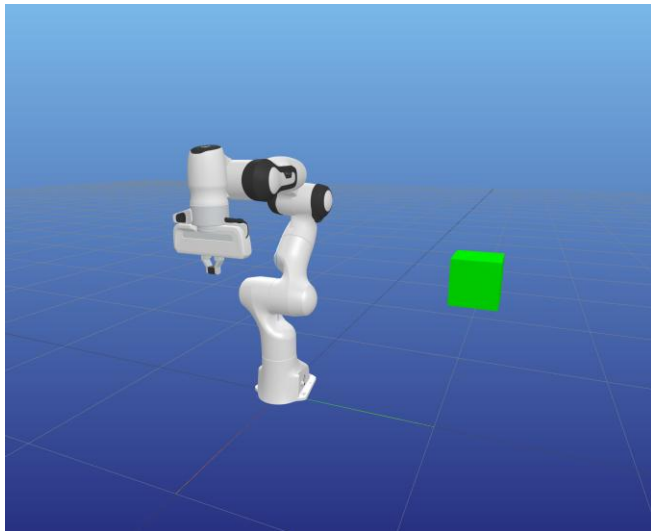


Optimal Control | Reaching target with a manipulator

Robot model setup (Pinocchio)

- ▶ The code begins by loading the Franka Emika Panda robot with `robex.load('panda')`
- ▶ Using `pin.buildReducedModel`, two gripper joints are frozen to create a reduced model, reducing unnecessary computation
- ▶ State x : the robot state includes not only the joint positions q but also the joint velocities v :

$$x = [q, v]^T \in \mathbb{R}^{14} \quad (7 \text{ joints} \times 2)$$



Optimal Control | Reaching target with a manipulator

Cost function design

Running cost: cost accumulated along the trajectory

Goal tracking: ResidualModelFrameTranslation

– 3D tracking cost: $\ell(x) = \| p(q) - p^* \|^2$

– 6D tracking cost: $\ell(x) = \| \log({}^0M_E(q)^{-1} {}^0M_*) \|^2$

State regularization: Encourages the state to remain near a reference

$$\| x - x^* \|^2$$

```
Terminal cost for state regularization || x - x* ||**2
xRegWeights = crocodyl.ActivationModelWeightedQuad(np.array([1,1,1,1,1,1,1,
                                                             .1,.1,.1,.1,.1,.1,.1]))
# Require more strictly a small velocity at task end and care less about position
xRegWeightsT=crocodyl.ActivationModelWeightedQuad(np.array([.5,.5,.5,.5,.5,.5,.5,
                                                             5.,5.,5.,5.,5.,5.,5.])))
```

Control regularization: Penalizes aggressive control inputs

$$\| u - g(q) \|^2$$

Optimal Control | Reaching target with a manipulator

■ Action models (differential and integrated)

► Differential Action Model (DAM)

Defines the dynamics and the cost function at continuous time.

$$\dot{x} = f(x, u)$$

► Integrated Action Model (IAM)

Converts the continuous dynamics into discrete time. In this example, Euler integration is used:

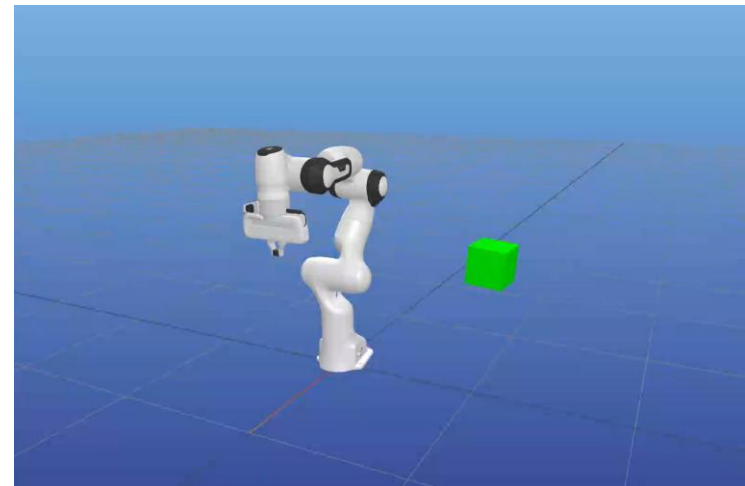
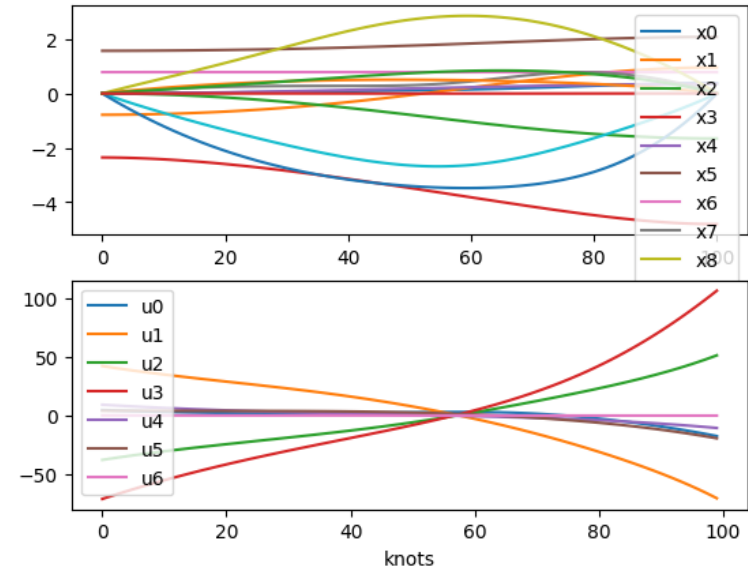
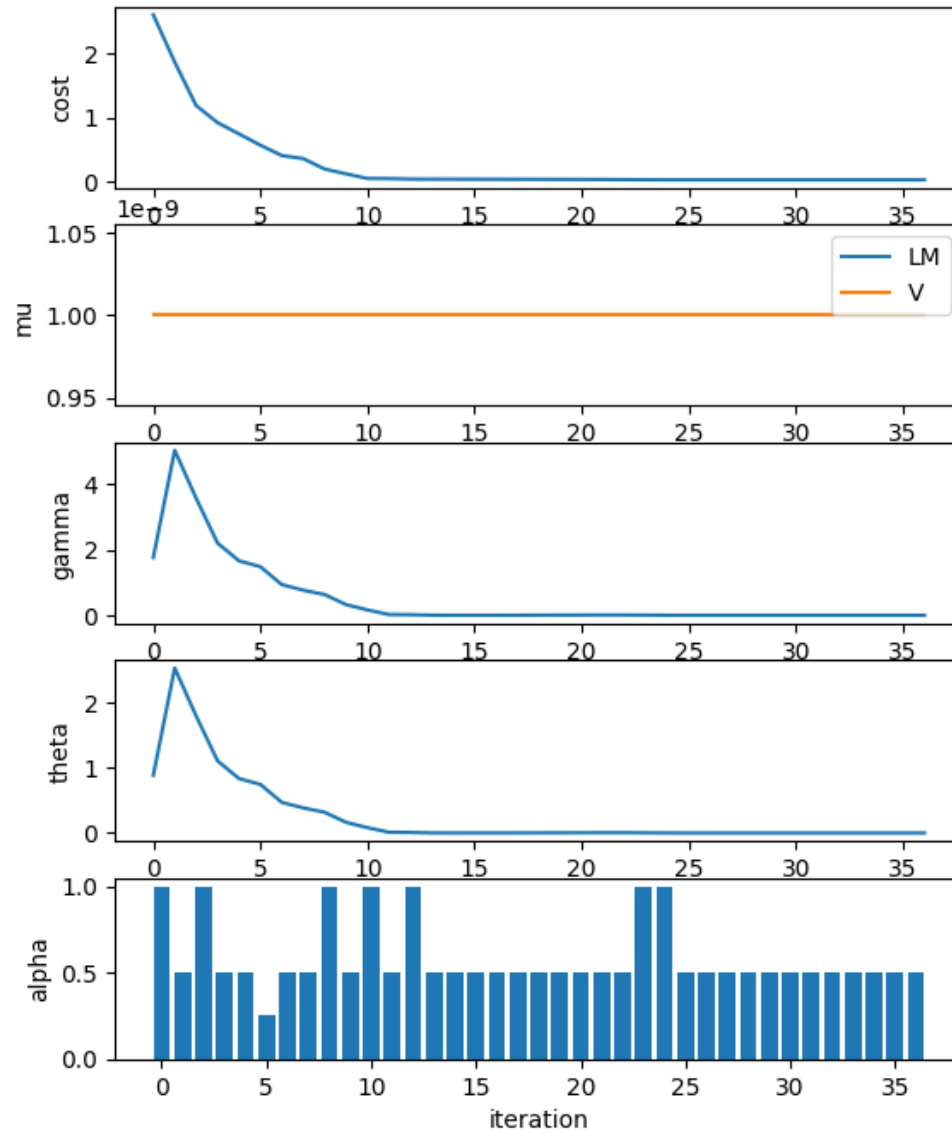
$$x_{k+1} = x_k + \dot{x}\Delta t$$

■ DDP Solver (Differential Dynamic Programming)

► Shooting problem

Starting from the initial state x_0 , a sequence of action models is traversed to generate a trajectory.

Optimal Control | Reaching target with a manipulator



Multi-target reaching

Logical structure

- Total horizon length: $T = T_1 + T_2 + T_3 + T_4$
- Sequence 1: $t = 0 \sim T_1$, target
- Sequence 2: $t = T_1 \sim T_2$, target
- Sequence 3, ...: defined similarly with their respective goals

```
seq0 = [runningModels[0]]*T + [terminalModels[0]]
seq1 = [runningModels[1]]*T + [terminalModels[1]]
seq2 = [runningModels[2]]*T + [terminalModels[2]]
seq3 = [runningModels[3]]*T
problem =
crocoddyl.ShootingProblem(robot_model.x0, seq0+seq1+seq2+seq3, terminalM
odels[3])
```

► Defining Action Models

```
runningModels = []  
terminalModels = []
```

for each waypoint create an `IntegratedActionModelEuler()` and append it to the list

► Adjusting the weights

At this stage, proper weight tuning is essential.

the weights I added:

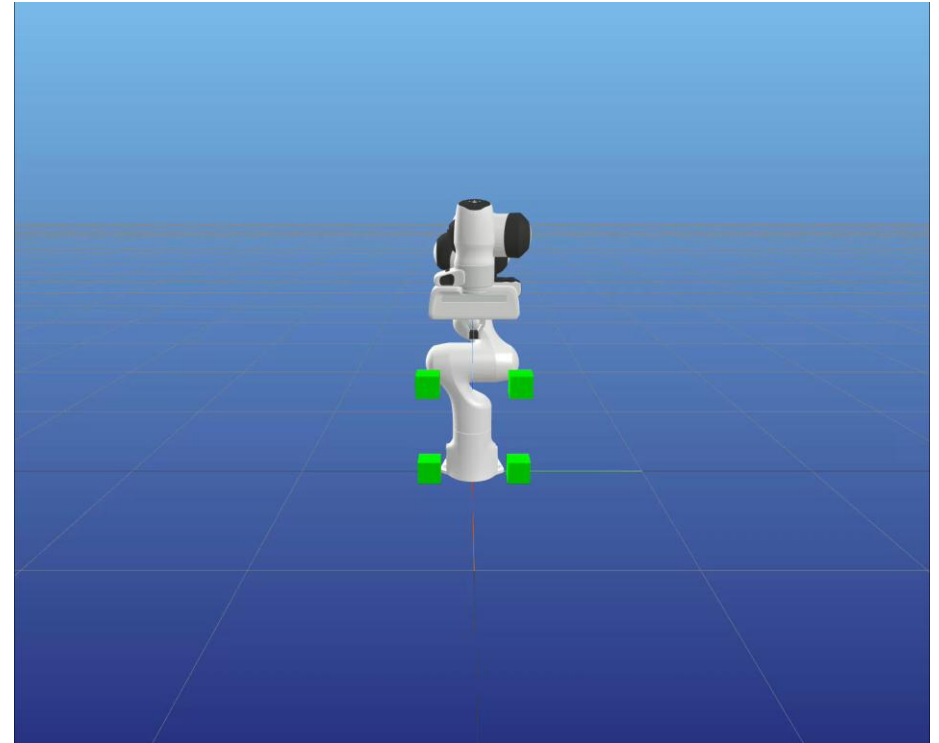
```
runningCostModel.addCost("gripperPose", goalTrackingCost, .1)  
terminalCostModel.addCost("gripperPose", goalTrackingCost, 4)  
runningCostModel.addCost("xReg", xRegCost, 1e-3)  
terminalCostModel.addCost("xReg", xRegCostT, .01)  
runningCostModel.addCost("uReg", uRegCost, 1e-6)
```

Optimal Control | Reaching multiple targets with a manipulator

```
target_positions = [  
    np.array([0.4, -0.3, 0.4]),  
    np.array([0.4, -0.1, 0.4]),  
    np.array([0.4, 0.1, 0.4]),  
    np.array([0.4, 0.3, 0.4]),  
]
```



```
target_positions = [  
    np.array([0.4, -0.1, 0.4]),  
    np.array([0.4, -0.1, 0.2]),  
    np.array([0.4, 0.1, 0.4]),  
    np.array([0.4, 0.1, 0.2]),  
]
```



Thank You